SPECIFICATION

Calypso Terminal API

Calypso Card

Version 3.0.0-SNAPSHOT, 2026-07-20

⚠ ***Warning. The present document cancels and replaces the previous release.***

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Link and Contact



Website
https://calypsonet.org



Support
support@calypsonet.org

Copyright © Calypso Networks Association, https://calypsonet.org.

This specification is licensed under the **Creative Commons Attribution-NoDerivatives 4.0 International** *(CC BY-ND 4.0).*

You are free to **share** *— copy and redistribute this document in any medium or format for any purpose, even commercially — under the following terms:*

- ***Attribution*** *— You MUST give appropriate credit to the Calypso Networks Association, provide a link to the license, and indicate if changes were made. You MAY do so in any reasonable manner, but not in any way that suggests the Calypso Networks Association endorses you or your use.*
- ***NoDerivatives*** *— If you remix, transform, or build upon this document, you MAY NOT distribute the modified material.*

No additional restrictions — You MAY NOT apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Implementation grant *— The* **NoDerivatives** *condition above applies to this specification* **document** *only — its text, tables and diagrams. It does* **not** *restrict the use of the technical information the document contains. The Calypso Networks Association hereby grants everyone an irrevocable, worldwide, royalty-free permission to use the information contained in this specification to design, develop, and distribute software implementations of this API, in any programming language, under the license of their choice.*

The full license text is available at https://creativecommons.org/licenses/by-nd/4.0/.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Table of Contents

1. Abstract . . . 8
2. Introduction . . . 9
2.1. Purpose . . . 9
2.2. Scope . . . 9
2.3. Conformance . . . 9
2.4. Document Conventions . . . 10
2.4.1. Naming . . . 10
2.4.2. Data types . . . 10
2.4.3. Type tables . . . 10
2.4.4. Operation tables . . . 11
2.4.5. Operation contracts . . . 11
2.4.6. Concurrency . . . 11
2.5. References and Resources . . . 12
2.5.1. Calypso References . . . 12
2.5.2. Normative References . . . 12
2.5.3. External Resources . . . 12
3. Glossary and Acronyms . . . 13
3.1. Glossary . . . 13
3.2. Acronyms . . . 13
4. Architectural Overview . . . 14
4.1. Functional positioning . . . 14
4.2. Logical namespaces . . . 14
4.3. Command exchange duration bounds . . . 15
4.3.1. Measured duration . . . 15
4.3.2. Resolution rule . . . 15
4.3.3. Behaviour on overrun . . . 15
4.3.4. Card serial number . . . 16
4.3.5. FCI matching . . . 16
4.3.6. FCI integrity . . . 16
4.3.7. Duplicates . . . 16
4.4. UML class diagram . . . 16
5. API Specification . . . 18
5.1. Constants . . . 18
5.1.1. Calypso Card API Properties . . . 18
5.2. Data . . . 18
5.2.1. Directory Header . . . 18
5.2.2. Elementary File . . . 18
5.2.3. File Header . . . 19
5.2.4. Search Command Data . . . 19
5.2.5. SV Debit Log Record . . . 20
5.2.6. SV Load Log Record . . . 20
5.3. API Interfaces . . . 21
5.3.1. Asymmetric Crypto Security Settings . . . 21
Add CA Certificate . . . 21
Add CA Certificate Parser . . . 21
Add Card Certificate Parser . . . 22
Add PCA Certificate . . . 22

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5.3.2. Calypso Card . . . 22
Get Application Serial Number . . . 23
Get Application Subtype . . . 23
Get Application Type . . . 23
Get CA Certificate . . . 23
Get Card Certificate . . . 24
Get Card Public Key . . . 24
Get Counter Values By LID . . . 24
Get Counter Values By SFI . . . 24
Get DF Name . . . 25
Get Directory Header . . . 25
Get File By LID . . . 25
Get File By SFI . . . 26
Get Files . . . 26
Get Matching Record Numbers . . . 26
Get PIN Attempt Remaining . . . 26
Get Platform . . . 27
Get Product Type . . . 27
Get Session Modification . . . 27
Get Software Issuer . . . 27
Get Software Revision . . . 27
Get Software Version . . . 28
Get Startup Info Raw Data . . . 28
Get SV Balance . . . 28
Get SV Debit Log All Records . . . 28
Get SV Debit Log All Records Raw Data . . . 29
Get SV Debit Log Last Record . . . 29
Get SV Debit Log Last Record Raw Data . . . 29
Get SV Last T Num . . . 29
Get SV Load Log Record . . . 30
Get SV Load Log Record Raw Data . . . 30
Get Traceability Information . . . 30
Get Transaction Counter . . . 31
Is DF Invalidated . . . 31
Is DF Ratified . . . 31
Is Extended Mode Supported . . . 31
Is HCE . . . 32
Is PIN Blocked . . . 32
Is PIN Feature Available . . . 32
Is PKI Mode Supported . . . 32
Is Ratification On Deselect Supported . . . 32
Is SV Feature Available . . . 33
5.3.3. Calypso Card API Factory . . . 33
Create Asymmetric Crypto Security Settings . . . 33
Create Calypso Card Selection Extension . . . 33
Create Free Transaction Manager . . . 34
Create Secure Extended Mode Transaction Manager . . . 34
Create Secure PKI Mode Transaction Manager . . . 34
Create Secure Regular Mode Transaction Manager . . . 35

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Create Symmetric Crypto Security Settings . . . 35
5.3.4. Calypso Card Selection Extension . . . 36
Accept Invalidated Card . . . 36
Prepare Get Data . . . 36
Prepare Pre Open Secure Session . . . 37
Prepare Read Binary . . . 37
Prepare Read Counter . . . 38
Prepare Read Record . . . 38
Prepare Select File By Control . . . 38
Prepare Select File By LID . . . 39
5.3.5. Free Transaction Manager . . . 39
5.3.6. Secure Extended Mode Transaction Manager . . . 39
Prepare Activate Encryption . . . 39
Prepare Deactivate Encryption . . . 40
Prepare Early Mutual Authentication . . . 40
5.3.7. Secure PKI Mode Transaction Manager . . . 41
Prepare Open Secure Session . . . 41
5.3.8. Secure Regular Mode Transaction Manager . . . 41
5.3.9. Secure Symmetric Crypto Transaction Manager . . . 41
Prepare Change Key . . . 41
Prepare Invalidate . . . 42
Prepare Open Secure Session . . . 42
Prepare Rehabilitate . . . 43
Prepare SV Debit . . . 43
Prepare SV Get . . . 43
Prepare SV Reload . . . 44
Prepare SV Undebit . . . 44
5.3.10. Secure Transaction Manager . . . 45
Get Crypto Extension . . . 45
Prepare Cancel Secure Session . . . 45
Prepare Close Secure Session . . . 45
5.3.11. Security Settings . . . 46
Assign Close Secure Session Max Duration By CSN . . . 46
Assign Close Secure Session Max Duration By FCI . . . 46
Assign Open Secure Session Max Duration By CSN . . . 47
Assign Open Secure Session Max Duration By FCI . . . 47
Authorize File Not Found Error . . . 48
Authorize Record Not Found Error . . . 48
5.3.12. Symmetric Crypto Security Settings . . . 48
Add Authorized Session Key . . . 48
Add Authorized SV Key . . . 49
Assign Default KIF . . . 49
Assign Default KVC . . . 49
Assign KIF . . . 50
Assign SV Command Max Duration By CSN . . . 50
Assign SV Command Max Duration By FCI . . . 50
Authorize SV Negative Balance . . . 51
Disable Read On Session Opening . . . 51
Enable Multiple Session . . . 51

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Enable PIN Plain Transmission . . . 51
Enable Ratification Mechanism . . . 52
Enable SV Load And Debit Log . . . 52
Init Crypto Context For Next Transaction . . . 52
Set PIN Modification Ciphering Key . . . 53
Set PIN Verification Ciphering Key . . . 53
5.3.13. Transaction Manager . . . 53
Get Exchanged APDUs . . . 54
Get Secure Session State . . . 54
Prepare Append Record . . . 54
Prepare Change PIN . . . 55
Prepare Check PIN Status . . . 55
Prepare Decrease Counter . . . 55
Prepare Decrease Counters . . . 56
Prepare Generate Asymmetric Key Pair . . . 56
Prepare Get Data . . . 56
Prepare Increase Counter . . . 57
Prepare Increase Counters . . . 57
Prepare Put Data . . . 58
Prepare Read Binary . . . 58
Prepare Read Counter . . . 59
Prepare Read Record . . . 59
Prepare Read Records . . . 60
Prepare Read Records Partially . . . 61
Prepare Search Records . . . 61
Prepare Select File By Control . . . 62
Prepare Select File By LID . . . 62
Prepare Set Counter . . . 62
Prepare SV Read All Logs . . . 63
Prepare Update Binary . . . 63
Prepare Update Record . . . 64
Prepare Verify PIN . . . 64
Prepare Write Binary . . . 65
Prepare Write Record . . . 65
5.4. SPI Interfaces . . . 65
5.4.1. Asymmetric Crypto Card Transaction Manager Factory . . . 65
5.4.2. CA Certificate . . . 66
5.4.3. CA Certificate Parser . . . 66
5.4.4. Card Certificate . . . 66
5.4.5. Card Certificate Parser . . . 66
5.4.6. Card Transaction Crypto Extension . . . 66
5.4.7. PCA Certificate . . . 67
5.4.8. Symmetric Crypto Card Transaction Manager Factory . . . 67
5.5. Enumerations . . . 67
5.5.1. Calypso Card Product Type . . . 67
5.5.2. Elementary File Type . . . 68
5.5.3. Get Data Tag . . . 68
5.5.4. Put Data Tag . . . 68
5.5.5. Secure Session State . . . 69

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5.5.6. Select File Control . . . 69
5.5.7. SV Operation . . . 70
5.5.8. Write Access Level . . . 70
5.6. Errors . . . 70
5.6.1. Card Signature Not Verifiable . . . 70
5.6.2. Crypto . . . 71
5.6.3. Crypto IO . . . 71
5.6.4. Inconsistent Data . . . 71
5.6.5. Invalid Card Signature . . . 71
5.6.6. Invalid Certificate . . . 72
5.6.7. Invalid PIN . . . 72
5.6.8. Session Buffer Overflow . . . 72
5.6.9. Unauthorized Key . . . 72

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

1. Abstract

This document is the official technical specification of the **Terminal Calypso Card API** standardised by the **Calypso Networks Association** (CNA). It defines, in a language-agnostic way (using a notation inspired by the Kotlin language), the interfaces, classes, enumerations, errors, structural relationships and behavioural requirements that any conforming implementation of the Terminal Calypso Card API MUST satisfy.

The Terminal Calypso Card API is the **application-facing** surface used by terminal applications to operate Calypso transit and ticketing cards. It builds on top of the **Terminal Reader API** (CNA-TR-API) and delegates every cryptographic operation to a symmetric (CNA-TCCS-API) or asymmetric (CNA-TCCA-API) crypto module.

Document Status

Reference	YYMMDD-SP-CNATerminalAPI-CalypsoCard
Short name	CNA-TCC-API
Version	3.0.0-SNAPSHOT
Revision date	2026-07-20
Editor	Calypso Networks Association
Source repository	https://github.com/calypsonet/calypsonet-terminal-calypso-card-api-spec
Reference license	Creative Commons Attribution-NoDerivatives 4.0 International (CC BY-ND 4.0)

The present document specifies the 3.0.0-SNAPSHOT of the Terminal Calypso Card API. It defines a single, coherent baseline against which conforming implementations are evaluated; the **Since** column indicates the version in which each member was introduced.

Revision List

This section lists the high-level changes per version. The complete, fine-grained changelog is maintained in the `CHANGELOG.md` file at the root of the source repository.

Version / Date	Modifications
v3.0.0-SNAPSHOT 2026-07-20	Initial edition of this document.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

2. Introduction

2.1. Purpose

The Terminal Calypso Card API defines the public surface used by terminal applications to:

- select a Calypso card through a CNA-TR-API selection scenario enriched with Calypso-specific commands;
- operate read-only transactions without involving a crypto module (**free** transaction);
- operate fully authenticated transactions using a **symmetric** crypto module (CNA-TCCS-API)—either in the **regular** mode (compatible with every Calypso product) or in the **extended** mode (additional features only available on Calypso Prime Extended);
- operate fully authenticated transactions using an **asymmetric** crypto module (CNA-TCCA-API) in **PKI** mode;
- expose a dynamic view of the card (`CalypsoCard`) updated throughout the transaction.

2.2. Scope

This specification covers:

- the factory used to instantiate the public types of the API (`CalypsoCardApiFactory`);
- the data model of a Calypso card (`CalypsoCard`, `ElementaryFile`, `FileHeader`, `DirectoryHeader`, Stored Value log records);
- the selection extension (`CalypsoCardSelectionExtension`);
- the family of transaction managers (Free, Secure Regular, Secure Extended, Secure PKI);
- the **symmetric** and **asymmetric** security settings carriers;
- the **Stored Value** operations and **Search Record Multiple** support;
- the SPIs implemented by **crypto modules** to plug into the secure transaction managers;
- the family of errors raised during a Calypso transaction.

The following topics are **out of scope**:

- the on-the-wire APDU dialog with the card (handled internally by the implementation);
- the cryptographic algorithms themselves—only the SPI contract is normative;
- the on-the-wire transport between the terminal and any remote ticketing system.

2.3. Conformance

A software product conforms to this specification if and only if:

1. it implements every interface and class defined in Chapter 5 with the operations and semantics prescribed in this document,
2. it raises errors exclusively of the types defined in Section 5.6 for the situations described,
3. it preserves the structural relationships described in Chapter 4.

Throughout this specification, the key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY** and **OPTIONAL** are to be interpreted as described in RFC 2119 and RFC 8174 when, and only when, they appear in all capitals.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

In conformance with RFC 2119, **SHALL** is equivalent to **MUST**; this specification uses **MUST** exclusively in order to avoid ambiguity.

2.4. Document Conventions

2.4.1. Naming

Type names follow `UpperCamelCase`; operation, parameter and enumeration-member names follow `lowerCamelCase` except for enumeration values which use `UPPER_SNAKE_CASE`. Names defined by this specification are reproduced as-is in the UML class diagram (Section 4.4).

2.4.2. Data types

The following type names, whose notation is inspired by the Kotlin language, are used in operation signatures and are mapped, in each binding, to the closest equivalent native type. Implementations MUST preserve the semantic intent rather than the syntactic form.

Type	Description	Typical bindings
`String`	Sequence of characters, UTF-8 encoded by default.	`String`, `string`, `str`.
`Boolean`	Two-state truth value.	`boolean`, `bool`.
`Byte`	Signed 8-bit value.	`byte`, `i8`.
`Short`	Signed 16-bit integer.	`short`, `Int16`, `i16`.
`Int`	Signed 32-bit integer.	`int`, `Int32`, `i32`.
`ByteArray`	Ordered sequence of octets.	`byte[]`, `[u8]`, `bytes`.
`List<T>`	Ordered collection of `T` values, may contain duplicates.	`List<T>`, `Vec<T>`.
`Set<T>`	Unordered collection of unique `T` values.	`Set<T>`, `HashSet<T>`.
`Map<K,V>`	Associative array binding keys of type `K` to values of type `V`.	`Map<K,V>`, `Dictionary<K,V>`, `HashMap<K,V>`.
`SortedMap<K,V>`	Associative array sorted by key.	`SortedMap<K,V>`, `BTreeMap<K,V>`.
`Any`	Value of any type, root of the type hierarchy.	`Object`, `Any`, `object`, `Box<dyn Any>`.
`T?`	Nullable value: either a `T` value or `null`.	`Integer`/`Short`/`Byte` (boxed), `Optional<T>`, `T?`, `Option<T>`.
`null`	Absence of value, the alternative held by a nullable type.	`null`, `nil`, `None`.
`Self`	Own type of the receiver: a fluent operation returns the instance it was called on.	`Self`, `T` (recursive generic).
`Unit`	Indicates that an operation returns no value.	`void`, `Unit`, `()`.

2.4.3. Type tables

Each interface, class, enumeration and error defined in this document is described by a table of the form:

Name	The type's normalized name.
Kind	The category of the type (interface, class, enumeration, error, etc.).
Namespace	The namespace in which the type is defined.
Extends	The type extended by this type, when applicable.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Implements	The interface implemented by this type, when applicable.
Since	The version of the API in which the type was introduced.
Purpose	A normative description of the type's responsibility.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

The **Purpose** row states, in one or two sentences, the responsibility of the type and the way an instance is obtained. It is meant to be scanned, not read: any content that requires structure or depth—rules, lists, notes, value tables or examples—is placed as prose below the table.

2.4.4. Operation tables

Each operation defined in this document is described by a table of the form:

Signature	The operation's signature.
Since	The version of the API in which the operation was introduced.
Description	A normative description of the operation's behaviour.
Parameters	A description of each input parameter.
Returns	A description of the returned value, if any.
Pre-conditions	The conditions the caller MUST satisfy before invoking the operation, each prefixed by its nature.
Errors	A list of error conditions a correct caller must handle, each mapped to a specific error type.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

2.4.5. Operation contracts

To avoid repetition, the following rules apply to every operation table in Chapter 5 and are therefore not restated for each operation:

- **Pre-conditions.** The **Pre-conditions** row states what the caller MUST guarantee before invoking the operation. Violating one is a contract error, never listed in the **Errors** row, which describes only the outcomes a correct caller must handle. Each entry is prefixed by its nature, which determines the error raised when the condition is not met:
  - **Argument**—an illegal argument error. Applies to any invalid argument, including values bounded by a card or SAM protocol.
  - **Range**—an index out of bounds error. Applies when the argument designates a position within a collection or memory image exposed by the API.
  - **State**—an illegal state error.
  - **Capability**—an unsupported operation error.
- **Non-`null` results.** Unless the return type is marked nullable (`T?`) or the **Returns** row states otherwise, an operation returns a non-`null` value.
- **Collections.** Unless the return type is marked nullable (`T?`), an operation whose return type is a collection (e.g. `List`, `Set`, `Map`) returns an empty collection rather than `null`.
- **Fluent composition.** Builder-style operations return the current instance so that calls can be chained.

2.4.6. Concurrency

Unless explicitly stated otherwise, the stateful types defined by this specification are **not** thread-safe. A given instance MUST be accessed from a single thread at a time; concurrent use of the same instance without

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

external synchronisation results in undefined behaviour. Distinct instances MAY be used concurrently.

2.5. References and Resources

2.5.1. Calypso References

References to CNA Terminal APIs designate the indicated **major** version; any later release within the same major is backward-compatible per that API's evolution policy.

Reference	Document
[CNA-TR-API]	*CNA Terminal API — Reader* (SP-CNATerminalAPI-Reader), version 3.0, Calypso Networks Association.
[CNA-TCCS-API]	*CNA Terminal API — Calypso Crypto Symmetric* (SP-CNATerminalAPI-CalypsoCryptoSymmetric), version 0.1, Calypso Networks Association.
[CNA-TCCA-API]	*CNA Terminal API — Calypso Crypto Asymmetric* (SP-CNATerminalAPI-CalypsoCryptoAsymmetric), version 0.2, Calypso Networks Association.

2.5.2. Normative References

Reference	Document
[RFC 2119]	*Key words for use in RFCs to Indicate Requirement Levels*, S. Bradner, March 1997.
[RFC 8174]	*Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*, B. Leiba, May 2017.
[ISO/IEC 7816]	*Identification cards — Integrated circuit cards*.

2.5.3. External Resources

Resource	Link
Calypso Networks Association	https://calypsonet.org
Terminal APIs website	https://terminal-api.calypsonet.org
Terminal APIs documentation	https://docs.terminal-api.calypsonet.org

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

3. Glossary and Acronyms

3.1. Glossary

Term	Definition
Anti-passback	Minimum delay imposed by the transport operator between two uses of the same card.
Calypso Card	Smart card compliant with the Calypso specifications.
Secure Session	Authenticated and integrity-protected exchange between the terminal and the card.
Regular Mode	Secure session mode compatible with every Calypso product.
Extended Mode	Secure session mode supporting additional features (early mutual authentication, encryption of session APDUs, larger sessions). Available on Calypso Prime Extended.
PKI Mode	Secure session mode using asymmetric cryptography (Calypso 3.3+).
Stored Value (SV)	Calypso feature exposing a card-resident monetary counter.
Crypto Module	Component implementing the SPI exposed by CNA-TCCS-API or CNA-TCCA-API.
KIF / KVC	Calypso Key Identifier / Key Version Code.
HCE	Host Card Emulation.

3.2. Acronyms

Abbreviation	Expansion
AID	Application IDentifier
APDU	Application Protocol Data Unit
CNA	Calypso Networks Association
CSN	Calypso Serial Number (Application Serial Number)
DF	Dedicated File
EF	Elementary File
FCI	File Control Information
FCP	File Control Parameters
HCE	Host Card Emulation
KIF	Key Identifier
KVC	Key Version Code
LID	Long IDentifier (file id)
MAC	Message Authentication Code
PIN	Personal Identification Number
PKI	Public Key Infrastructure
SAM	Secure Application Module
SFI	Short File Identifier
SPI	Service Provider Interface
SV	Stored Value
UML	Unified Modelling Language

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

4. Architectural Overview

4.1. Functional positioning

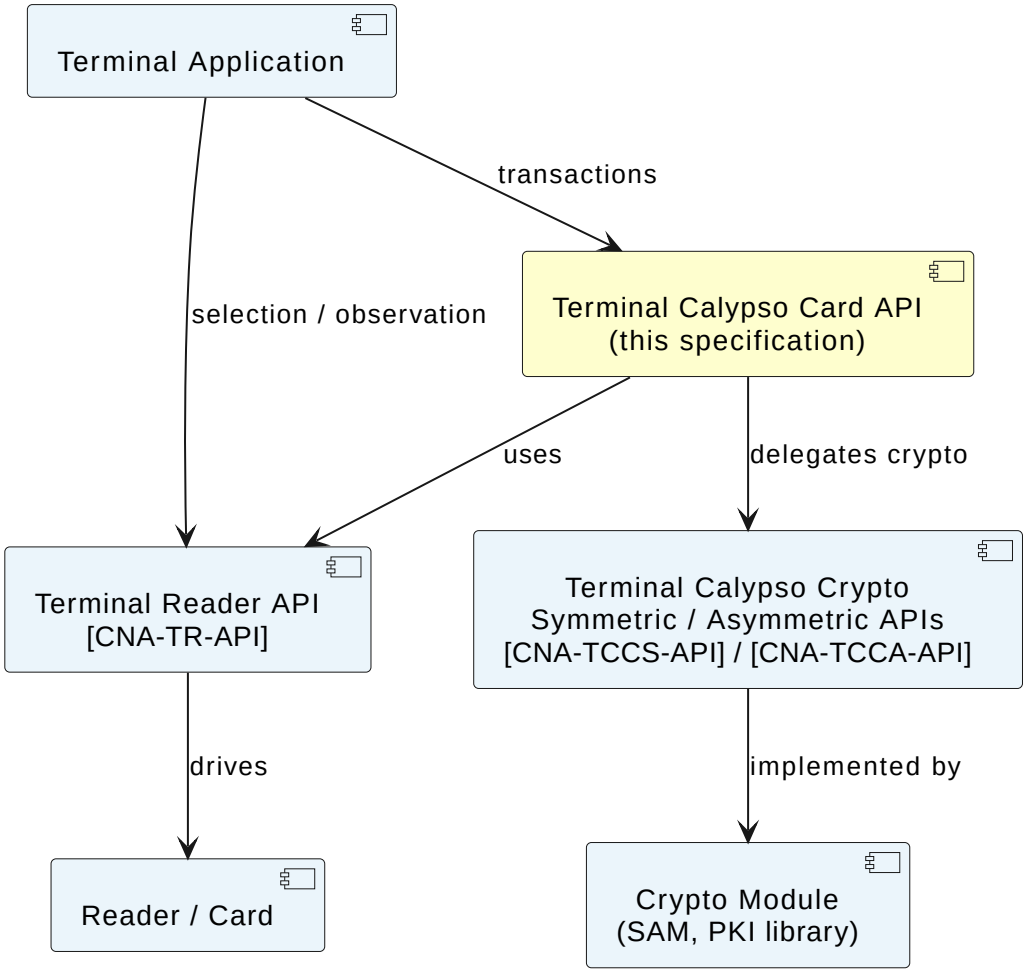


Figure 1. Terminal Calypso Card API — Architecture Overview

4.2. Logical namespaces

Namespace	Role	Summary
`calypso.card`	Public API	Factory, properties, enumerations (`GetDataTag`, `PutDataTag`, `SelectFileControl`, `WriteAccessLevel`).
`calypso.card.card`	Public API	Data model of a Calypso card: `CalypsoCard`, files, headers, SV logs and selection extension.
`calypso.card.transaction`	Public API	Transaction managers (Free / Secure Regular / Secure Extended / Secure PKI), security settings carriers, `SearchCommandData`, `SvOperation` and the family of errors.
`calypso.card.transaction.spi`	SPI	Marker interfaces implemented by crypto extensions (`SymmetricCryptoCardTransactionManagerFactory`, `AsymmetricCryptoCardTransactionManagerFactory`, certificate markers, `CardTransactionCryptoExtension`).

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

4.3. Command exchange duration bounds

As a countermeasure against relay attacks, the security settings allow the integrator to bound the duration of the most sensitive command exchanges: a relay adds a transmission delay, so that an abnormally long exchange can reveal that the card is not actually present in front of the reader. The bounds are declared through the `assign…MaxDuration…` operations of `SecuritySettings`, the parent of `SymmetricCryptoSecuritySettings` and `AsymmetricCryptoSecuritySettings`, each available in a CSN-based (`…ByCsn`) and an FCI-based (`…ByFci`) form.

4.3.1. Measured duration

Each bound applies to a single command exchange, measured from the transmission of the command to the reception of its response; the other commands of the secure session or of the Stored Value operation are not included. The bounded exchanges are:

- the **Open Secure Session** command (`assignOpenSecureSessionMaxDuration…`);
- the **Close Secure Session** command (`assignCloseSecureSessionMaxDuration…`);
- the **SV Reload**, **SV Debit** or **SV Undebit** command (`assignSvCommandMaxDuration…`, symmetric security settings only).

Durations are expressed in microseconds. The effective resolution of the measurement depends on the reader and on the execution platform, and a bound finer than that resolution cannot be relied upon.

The settings are recorded separately for each security settings instance and for each kind of bounded command exchange.

4.3.2. Resolution rule

For a given card, the applicable bound is determined as follows:

1. **CSN-based settings.** Each `…ByCsn(…)` call defines a **range** bounded by its `csnMin` and the immediately higher declared `csnMin` (or `+∞` for the highest threshold). If the card's CSN belongs to a range and the `maxDuration` of that range is not `Long.MAX_VALUE`, this value applies and the resolution stops.
2. **FCI-based settings.** Otherwise, the `…ByFci(…)` settings are evaluated in their declaration order and the first one whose `fciRegex` matches the card's FCI applies; a `maxDuration` equal to `Long.MAX_VALUE` then means that no bound applies.
3. Otherwise, no bound applies.

CSN-based settings therefore act as **overrides** of the FCI-based settings. A default bound for every card is expressed by a last FCI-based setting with the expression `.*`, not by a CSN-based setting with the lowest `csnMin`, which would shadow every FCI-based setting. Assigning `Long.MAX_VALUE` to a CSN range hands the decision back to the FCI-based settings for the cards of that range. The CSN-based mechanism allows the integrator to introduce or tighten duration bounds progressively as new generations of cards (with higher CSNs) are issued.

4.3.3. Behaviour on overrun

When the effective duration of a bounded command exchange exceeds the applicable bound, the exchange is reported as failed and the error is propagated to the application as an `InvalidCardResponse` (CNA-TR-API).

- **A secure session is open.** The session MUST be cancelled automatically before the error is propagated, so that no modification performed during the session is validated by the card.
- **No secure session is open** (a Stored Value command executed outside a session). There is nothing to

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

cancel: only the error is propagated, and the ticketing application decides what to do according to its own context (for example retrying the operation, rejecting the card or raising an alert).

4.3.4. Card serial number

The card serial number (CSN) is the Application Serial Number (see `getApplicationSerialNumber()`) interpreted as an unsigned 64-bit big-endian integer; `csnMin` is interpreted as the same unsigned 64-bit value, so that the comparison is unsigned.

4.3.5. FCI matching

- The FCI is the data returned by the **Select Application** command (see `IsoSmartCard.getSelectApplicationResponse()` in CNA-TR-API), excluding the status word. When no FCI is available, no FCI-based setting matches.
- The FCI is represented as an uppercase hexadecimal string without separators (two characters per byte).
- The match is complete: the whole string MUST match the expression, as if it were anchored at both ends. Since `.` matches a single hexadecimal digit, a byte is matched by `..`.
- To remain portable across implementation languages, the expression MUST be restricted to the following subset: literal characters, `.`, character classes `[…]` (including ranges and negation), the quantifiers `*`, `+`, `?`, `{n}`, `{n,}` and `{n,m}`, the alternation `|` and the groups `(…)`. Backreferences, lookaround assertions, anchors and inline flags are excluded.

4.3.6. FCI integrity

The FCI is obtained during the card selection, outside any secure session, and is therefore not authenticated. When an expression filters on the startup information data contained in the FCI, the integrator MUST schedule a `prepareGetData(FCI_FOR_CURRENT_DF)` inside the secure session, so that the integrity of the FCI obtained during the selection is guaranteed by the session: the following `processCommands()` raises `InconsistentData` if the returned FCI differs from the one obtained during the selection.

4.3.7. Duplicates

A `…ByCsn(…)` call with an already declared `csnMin` replaces the previous value. A `…ByFci(…)` call with an already declared `fciRegex` (identical character string) replaces the previous value and keeps its position in the evaluation order.

4.4. UML class diagram

The following UML class diagram provides a visual representation of the types and relationships defined by this specification:

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

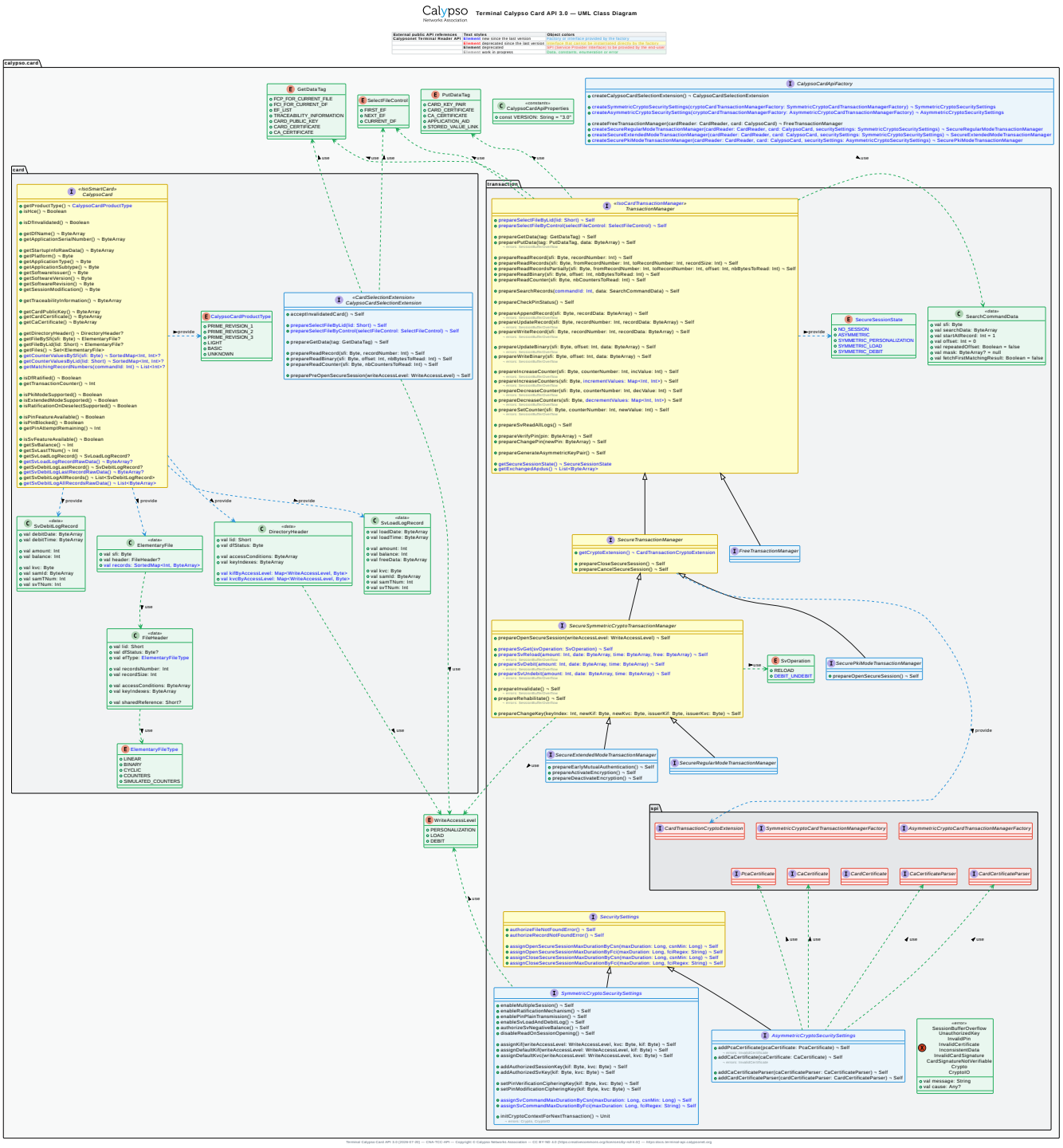


Figure 2. Terminal Calypso Card API — UML Class Diagram

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5. API Specification

5.1. Constants

5.1.1. Calypso Card API Properties

Name	`CalypsoCardApiProperties`
Kind	Constants
Namespace	`calypso.card`
Since	1.0
Purpose	Exposes the immutable properties of the Terminal Calypso Card API.

Name	Type	Since	Description
`VERSION`	`String`	1.0	String representation of the version of the API implemented by the conforming binding. The value is a dotted decimal of the form `MAJOR.MINOR`; it is `"3.0"` for this revision of the specification.

5.2. Data

5.2.1. Directory Header

Name	`DirectoryHeader`
Kind	Data
Namespace	`calypso.card.card`
Since	1.0
Purpose	Carries the metadata of a Calypso DF.

Name	Type	Default	Since	Description
`lid`	`Short`	—	1.0	LID, the 2-byte identifier of the associated file structure (DF or EF).
`dfStatus`	`Byte`	—	1.0	Status byte of the DF (Dedicated File) described by this header.
`accessConditions`	`ByteArray`	—	1.0	Access conditions of the DF. Non-empty.
`keyIndexes`	`ByteArray`	—	1.0	Key indexes of the DF. Non-empty.
`kifByAccessLevel`	`Map<WriteAccessLevel, Byte>`	—	1.0	KIF of each write access level, indexed by `WriteAccessLevel`. Contains one entry per write access level.
`kvcByAccessLevel`	`Map<WriteAccessLevel, Byte>`	—	1.0	KVC of each write access level, indexed by `WriteAccessLevel`. Contains one entry per write access level.

5.2.2. Elementary File

Name	`ElementaryFile`
Kind	Data
Namespace	`calypso.card.card`

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Since	1.0
Purpose	Calypso Elementary File. Carries an `SFI`, a `FileHeader` and the content of its records.

Name	Type	Default	Since	Description
`sfi`	`Byte`	—	1.0	Short File Identifier (SFI) of this Elementary File.
`header`	`FileHeader` ?	—	1.0	Metadata of the file. `null` when not yet set.
`records`	`SortedMap< Int, ByteArray>`	—	3.0	Known content of every record, indexed by record number. For a **binary** file, the whole content is carried by record #1. Empty when no record content is known; a record absent from the map has not been read.

5.2.3. File Header

Name	`FileHeader`
Kind	Data
Namespace	`calypso.card.card`
Since	1.0
Purpose	Carries the metadata of a Calypso EF.

Name	Type	Default	Since	Description
`lid`	`Short`	—	1.0	LID, the 2-byte identifier of the associated file structure (DF or EF).
`dfStatus`	`Byte?`	—	1.0	DF status. `null` when not available—for example when the header is built from the response to a **Get Data** with the `EF_LIST` tag.
`efType`	`ElementaryFileType`	—	1.0	Type of this Elementary File (EF).
`recordsNumber`	`Int`	—	1.0	Number of records. Always `1` for a **counter** file and for a **binary** file. For a counter file, the extra bytes—the remainder of the division of the file size by 3—are not accessible.
`recordSize`	`Int`	—	1.0	Size of a record. For a **counter** file, the original size of record #1; the extra bytes—the remainder of the division of the file size by 3—are not accessible. For a **binary** file, the size of the file.
`accessConditions`	`ByteArray`	—	1.0	Access conditions. Empty when not available.
`keyIndexes`	`ByteArray`	—	1.0	Key indexes. Empty when not available.
`sharedReference`	`Short?`	—	1.0	Non-zero unique identifier of the shared data when the file data is shared, `0` when it is not shared. `null` when the information is not available.

5.2.4. Search Command Data

Name	`SearchCommandData`
Kind	Data
Namespace	`calypso.card.transaction`
Since	1.1
Purpose	Carries the search criteria of `TransactionManager.prepareSearchRecords`, built directly by the application.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Name	Type	Default	Since	Description
`sfi`	`Byte`	—	1.1	SFI of the EF in which the search is performed.
`searchData`	`ByteArray`	—	1.1	Byte sequence to search for within the targeted record.
`startAtRecord`	`Int`	`1`	1.1	Record number at which the search begins.
`offset`	`Int`	`0`	1.1	Offset, in bytes, at which the analysis starts within each record.
`repeatedOffset`	`Boolean`	`false`	1.1	When true, the data is analysed at `offset`, then repeatedly at each following offset until the end of the record.
`mask`	`ByteArray?`	`null`	1.1	Comparison mask specifying precisely the bits to take into account in the comparison. Its length MUST be ⇐ the length of `searchData`; missing bytes are right-padded with `FFh`. `null` applies no mask.
`fetchFirstMatchingResult`	`Boolean`	`false`	1.1	When true, the content of the first matching record is fetched into the `CalypsoCard`.

5.2.5. SV Debit Log Record

Name	`SvDebitLogRecord`
Kind	Data
Namespace	`calypso.card.card`
Since	1.0
Purpose	One record of a Stored Value debit log.

Name	Type	Default	Since	Description
`debitDate`	`ByteArray`	—	1.0	2-byte debit date. Non-empty.
`debitTime`	`ByteArray`	—	1.0	2-byte debit time. Non-empty.
`amount`	`Int`	—	1.0	Debit amount.
`balance`	`Int`	—	1.0	SV balance after the debit.
`kvc`	`Byte`	—	1.0	KVC of the debit key.
`samId`	`ByteArray`	—	1.0	4-byte SAM ID. Non-empty.
`samTNum`	`Int`	—	1.0	SAM transaction number.
`svTNum`	`Int`	—	1.0	SV transaction number.

5.2.6. SV Load Log Record

Name	`SvLoadLogRecord`
Kind	Data
Namespace	`calypso.card.card`
Since	1.0
Purpose	One record of a Stored Value load log.

Name	Type	Default	Since	Description
`loadDate`	`ByteArray`	—	1.0	2-byte load date. Non-empty.
`loadTime`	`ByteArray`	—	1.0	2-byte load time. Non-empty.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Name	Type	Default	Since	Description
`amount`	`Int`	—	1.0	Load amount.
`balance`	`Int`	—	1.0	SV balance after the load.
`freeData`	`ByteArray`	—	1.0	2-byte free data. Non-empty.
`kvc`	`Byte`	—	1.0	KVC of the load key.
`samId`	`ByteArray`	—	1.0	4-byte SAM ID. Non-empty.
`samTNum`	`Int`	—	1.0	SAM transaction number.
`svTNum`	`Int`	—	1.0	SV transaction number.

5.3. API Interfaces

5.3.1. Asymmetric Crypto Security Settings

Name	`AsymmetricCryptoSecuritySettings`
Kind	Interface
Namespace	`calypso.card.transaction`
Extends	`SecuritySettings`
Since	2.1
Purpose	Security settings carrier for PKI Calypso card transactions. An instance is obtained via `CalypsoCardApiFactory.createAsymmetricCryptoSecuritySettings`.

Add CA Certificate

Signature	`addCaCertificate(`
	`  caCertificate: CaCertificate`
	`) → Self`

Since	2.1
Description	Registers a CA certificate. The issuer's certificate MUST be loaded first. Preloading a CA certificate avoids having to read it from the card. Various checks are performed to ensure the integrity and validity of the supplied CA certificate: verification of the certificate's signature using the issuer's public key, check of the validity period to ensure the certificate is neither expired nor prematurely valid, confirmation of the authenticity of the issuer and subject details, and compliance with any constraints or extensions required for CA certificates.
Parameters	`caCertificate` (`CaCertificate`) — the CA certificate to register.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **State** — the contained public key is not already registered.
Errors	`InvalidCertificate` — if the check of the supplied certificate failed.
See also	`addPcaCertificate`

Add CA Certificate Parser

Signature	`addCaCertificateParser(`
	`  caCertificateParser: CaCertificateParser`
	`) → Self`

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Since	2.1
Description	Registers a CA certificate parser, used when the CA certificate is not already available. The parser provides the means to build a CA certificate from the raw data read from the card.
Parameters	`caCertificateParser` (`CaCertificateParser`) — the CA certificate parser to register.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **State** — no parser is registered for the same certificate type.
Errors	None.

Add Card Certificate Parser

Signature	`addCardCertificateParser(`
	`  cardCertificateParser: CardCertificateParser`
	`) → Self`

Since	2.1
Description	Registers a card certificate parser. The parser provides the means to build a card certificate from the raw data read from the card. Only one parser MAY be registered per certificate type.
Parameters	`cardCertificateParser` (`CardCertificateParser`) — the card certificate parser to register.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **State** — no parser is registered for the same certificate type.
Errors	None.

Add PCA Certificate

Signature	`addPcaCertificate(`
	`  pcaCertificate: PcaCertificate`
	`) → Self`

Since	2.1
Description	Registers a self-signed PCA certificate. Various checks are performed to ensure the integrity and validity of the supplied PCA certificate: verification of the certificate's signature to ensure it is self-signed, check of the validity period to ensure the certificate is neither expired nor prematurely valid, confirmation of the authenticity of the issuer and subject details, and compliance with any constraints or extensions required for PCA certificates.
Parameters	`pcaCertificate` (`PcaCertificate`) — the self-signed PCA certificate to register.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — the certificate is valid. **State** — the contained public key is not already registered.
Errors	`InvalidCertificate` — if the check of the supplied certificate failed.

5.3.2. Calypso Card

Name	`CalypsoCard`
Kind	Interface
Namespace	`calypso.card.card`
Extends	`IsoSmartCard` (CNA-TR-API)

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Since	1.0
Purpose	Dynamic view of the card's content, updated from selection to the end of the transaction.

An instance of `CalypsoCard` is obtained by casting the `IsoSmartCard` object produced by the selection process defined in CNA-TR-API.

A `CalypsoCard` carries: application identification (revision, class, DF name, serial number, ATR, issuer), the indication of optional features (Stored Value, PIN, Rev 3.2 mode, ratification management), the management information of the modification buffer, the invalidation status and the files / counters / SV data read or modified during the transaction.

The `CalypsoCard` instance bound to a transaction is a **dynamic view** of the card image, updated after every successful command. Failed write commands do not update the in-memory image.

Get Application Serial Number

Signature	`getApplicationSerialNumber() → ByteArray`
Since	1.0
Description	Returns the Application Serial Number, an 8-byte value uniquely identifying the card application.
Returns	A non-empty `ByteArray`.
Pre-conditions	None.
Errors	None.

Get Application Subtype

Signature	`getApplicationSubtype() → Byte`
Since	1.0
Description	Returns the application subtype byte, which references the card's file structure.
Returns	A `Byte` value.
Pre-conditions	None.
Errors	None.

Get Application Type

Signature	`getApplicationType() → Byte`
Since	1.0
Description	Returns the application type byte, which encodes the product type together with its options.
Returns	A `Byte` value.
Pre-conditions	None.
Errors	None.

Get CA Certificate

Signature	`getCaCertificate() → ByteArray`
Since	2.1
Description	CA certificate; empty if not available.
Returns	A `ByteArray`; empty if not available.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Pre-conditions	None.
Errors	None.

See also	`CalypsoCardSelectionExtension.prepareGetData`
	`TransactionManager.prepareGetData`
	`SecurePkiModeTransactionManager.prepareOpenSecureSession`

Get Card Certificate

Signature	`getCardCertificate() → ByteArray`
Since	2.1
Description	Card certificate; empty if not available.
Returns	A `ByteArray`; empty if not available.
Pre-conditions	None.
Errors	None.

See also	`CalypsoCardSelectionExtension.prepareGetData`
	`TransactionManager.prepareGetData`
	`SecurePkiModeTransactionManager.prepareOpenSecureSession`

Get Card Public Key

Signature	`getCardPublicKey() → ByteArray`
Since	2.1
Description	Card public key; empty if not available.
Returns	A `ByteArray`; empty if not available.
Pre-conditions	None.
Errors	None.

See also	`CalypsoCardSelectionExtension.prepareGetData`
	`TransactionManager.prepareGetData`
	`SecurePkiModeTransactionManager.prepareOpenSecureSession`

Get Counter Values By LID

Signature	`getCounterValuesByLid(lid: Short) → SortedMap<Int, Int>?`
Since	3.0
Description	Values of the counters carried by the EF whose LID matches, indexed by counter number. Each value is decoded from the 3-byte slice at offset `(n - 1) * 3` of record #1, for both `COUNTERS` and `SIMULATED_COUNTERS` files; a trailing slice shorter than 3 bytes yields no entry.
Parameters	`lid` — the LID of the EF whose counter values are requested.
Returns	A `SortedMap<Int, Int>`, empty if record #1 of that EF has not been read yet. `null` if no EF with this LID is known, or if its type carries no counters.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Get Counter Values By SFI

Signature	`getCounterValuesBySfi(sfi: Byte) → SortedMap<Int, Int>?`
Since	3.0

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Values of the counters carried by the EF whose SFI matches, indexed by counter number. Each value is decoded from the 3-byte slice at offset `(n - 1) * 3` of record #1, for both `COUNTERS` and `SIMULATED_COUNTERS` files; a trailing slice shorter than 3 bytes yields no entry.
Parameters	`sfi` — the SFI of the EF whose counter values are requested.
Returns	A `SortedMap<Int, Int>`, empty if record #1 of that EF has not been read yet. `null` if no EF with this SFI is known, or if its type carries no counters.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Get DF Name

Signature	`getDfName() → ByteArray`
Since	1.0
Description	DF name (5 to 16 bytes), as per ISO/IEC 7816-4. It also corresponds to the complete representation of the target covered by the AID value supplied in the selection command: the AID selects the application by specifying all or part of the targeted DF name (5 bytes minimum).
Returns	A non-empty `ByteArray`.
Pre-conditions	None.
Errors	None.

Get Directory Header

Signature	`getDirectoryHeader() → DirectoryHeader?`
Since	1.0
Description	Metadata of the current DF; `null` if not set.
Returns	A `DirectoryHeader`, or `null` if not set.
Pre-conditions	None.
Errors	None.

See also	`CalypsoCardSelectionExtension.prepareSelectFileByLid`
	`CalypsoCardSelectionExtension.prepareSelectFileByControl`
	`TransactionManager.prepareSelectFileByLid`
	`TransactionManager.prepareSelectFileByControl`

Get File By LID

Signature	`getFileByLid(lid: Short) → ElementaryFile?`
Since	1.0
Description	EF whose LID matches; `null` if not found. Note that when a secure session is running, the returned object carries all in-session modifications, which are rolled back if the secure session fails.
Parameters	`lid` — the LID to search.
Returns	An `ElementaryFile`, or `null` if not found.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Get File By SFI

Signature	`getFileBySfi(sfi: Byte) → ElementaryFile?`
Since	1.0
Description	EF whose SFI matches; `null` if not found, or if `sfi == 0`. Note that when a secure session is running, the returned object carries all in-session modifications, which are rolled back if the secure session fails.
Parameters	`sfi` — the SFI to search.
Returns	An `ElementaryFile`, or `null` if not found, or if `sfi == 0`.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Get Files

Signature	`getFiles() → Set<ElementaryFile>`
Since	1.1
Description	All known EFs of the current DF. During a secure session, this set reflects every in-session modification (which is rolled back on session failure).
Returns	A `Set<ElementaryFile>`; empty if no EF is known.
Pre-conditions	None.
Errors	None.

Get Matching Record Numbers

Signature	`getMatchingRecordNumbers(commandId: Int) → List<Int>?`
Since	3.0
Description	Numbers of the records that matched the search command prepared with this identifier. When several search commands share the same identifier, the numbers returned are those of the most recently processed one.
Parameters	`commandId` — the identifier attached to the search command when it was prepared.
Returns	A `List<Int>`, empty if the search matched no record. `null` if no search command carrying this identifier has been processed.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.
See also	`TransactionManager.prepareSearchRecords`

Get PIN Attempt Remaining

Signature	`getPinAttemptRemaining() → Int`
Since	1.0
Description	Returns the number of PIN presentation attempts still remaining before the PIN becomes blocked.
Returns	An `Int` value.
Pre-conditions	**State** — the PIN has been checked.
Errors	None.
See also	`TransactionManager.prepareCheckPinStatus` `TransactionManager.prepareVerifyPin`

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Get Platform

Signature	`getPlatform() → Byte`
Since	1.0
Description	Returns the platform identification byte, which references the card's chip.
Returns	A `Byte` value.
Pre-conditions	None.
Errors	None.

Get Product Type

Signature	`getProductType() → CalypsoCardProductType`
Since	1.0
Description	Returns the card product type (`PRIME_REVISION_1` / `PRIME_REVISION_2` / `PRIME_REVISION_3` / `LIGHT` / `BASIC` / `UNKNOWN`).
Returns	A `CalypsoCardProductType`.
Pre-conditions	None.
Errors	None.

Get Session Modification

Signature	`getSessionModification() → Byte`
Since	1.0
Description	Returns the session modification byte, which—depending on the card—caps either the number of bytes modifiable or the number of write commands allowed within a secure session.
Returns	A `Byte` value.
Pre-conditions	None.
Errors	None.

Get Software Issuer

Signature	`getSoftwareIssuer() → Byte`
Since	1.0
Description	Returns the software issuer byte, identifying the issuer of the card's embedded software.
Returns	A `Byte` value.
Pre-conditions	None.
Errors	None.

Get Software Revision

Signature	`getSoftwareRevision() → Byte`
Since	1.0
Description	Returns the software revision byte of the card's embedded software.
Returns	A `Byte` value.
Pre-conditions	None.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Errors	None.

Get Software Version

Signature	`getSoftwareVersion() → Byte`
Since	1.0
Description	Returns the software version byte of the card's embedded software.
Returns	A `Byte` value.
Pre-conditions	None.
Errors	None.

Get Startup Info Raw Data

Signature	`getStartupInfoRawData() → ByteArray`
Since	1.0
Description	Returns the raw Calypso startup information block—the sequence of bytes from which the individual startup fields (platform, application type and subtype, software issuer, version and revision, etc.) are derived.
Returns	A non-empty `ByteArray`.
Pre-conditions	None.
Errors	None.

Get SV Balance

Signature	`getSvBalance() → Int`
Since	1.0
Description	Returns the current Stored Value (SV) balance held by the card, expressed in its monetary unit.
Returns	An `Int` value.
Pre-conditions	**State**—an **SV Get** command has been executed.
Errors	None.

See also	`SecureSymmetricCryptoTransactionManager.prepareSvGet`
	`SecureSymmetricCryptoTransactionManager.prepareSvDebit`
	`SecureSymmetricCryptoTransactionManager.prepareSvReload`
	`SecureSymmetricCryptoTransactionManager.prepareSvUndebit`

Get SV Debit Log All Records

Signature	`getSvDebitLogAllRecords() → List<SvDebitLogRecord>`
Since	1.0
Description	All SV debit log records read from the card. Each call returns snapshots decoded from the values returned by `getSvDebitLogAllRecordsRawData` at the time of the call, in the same order.
Returns	A `List<SvDebitLogRecord>`; empty if no debit log record has been read.
Pre-conditions	None.
Errors	None.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

See also	`getSvDebitLogAllRecordsRawData`
	`SecureSymmetricCryptoTransactionManager.prepareSvGet`
	`SecureSymmetricCryptoTransactionManager.prepareSvDebit`
	`SecureSymmetricCryptoTransactionManager.prepareSvReload`
	`SecureSymmetricCryptoTransactionManager.prepareSvUndebit`

Get SV Debit Log All Records Raw Data

Signature	`getSvDebitLogAllRecordsRawData() → List<ByteArray>`
Since	3.0
Description	Returns the raw bytes of all SV debit log records read from the card. The list returned by `getSvDebitLogAllRecords` at the same time has the same size and order, each `SvDebitLogRecord` being the decoding of the raw record at the same position.
Returns	A `List<ByteArray>` of non-empty raw records; empty if no debit log record has been read.
Pre-conditions	None.
Errors	None.
See also	`getSvDebitLogAllRecords`

Get SV Debit Log Last Record

Signature	`getSvDebitLogLastRecord() → SvDebitLogRecord?`
Since	1.0
Description	Last SV debit log record; `null` if not available. Each call returns a snapshot decoded from the value returned by `getSvDebitLogLastRecordRawData` at the time of the call.
Returns	A `SvDebitLogRecord`, or `null` if not available.
Pre-conditions	None.
Errors	None.

See also	`getSvDebitLogLastRecordRawData`
	`SecureSymmetricCryptoTransactionManager.prepareSvGet`
	`SecureSymmetricCryptoTransactionManager.prepareSvDebit`
	`SecureSymmetricCryptoTransactionManager.prepareSvReload`
	`SecureSymmetricCryptoTransactionManager.prepareSvUndebit`

Get SV Debit Log Last Record Raw Data

Signature	`getSvDebitLogLastRecordRawData() → ByteArray?`
Since	3.0
Description	Returns the raw bytes of the last SV debit log record; `null` if not available. The `SvDebitLogRecord` returned by `getSvDebitLogLastRecord` at the same time is the decoding of this value; both are `null` together.
Returns	The non-empty raw bytes of the record, or `null` if not available.
Pre-conditions	None.
Errors	None.
See also	`getSvDebitLogLastRecord`

Get SV Last T Num

Signature	`getSvLastTNum() → Int`

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Since	1.0
Description	Returns the number of the last Stored Value (SV) transaction recorded on the card.
Returns	An `Int` value.
Pre-conditions	**State** — an **SV Get** command has been executed.
Errors	None.

See also	`SecureSymmetricCryptoTransactionManager.prepareSvGet`
	`SecureSymmetricCryptoTransactionManager.prepareSvDebit`
	`SecureSymmetricCryptoTransactionManager.prepareSvReload`
	`SecureSymmetricCryptoTransactionManager.prepareSvUndebit`

Get SV Load Log Record

Signature	`getSvLoadLogRecord() → SvLoadLogRecord?`
Since	1.0
Description	Last SV load log record; `null` if not available. Each call returns a snapshot decoded from the value returned by `getSvLoadLogRecordRawData` at the time of the call.
Returns	A `SvLoadLogRecord`, or `null` if not available.
Pre-conditions	None.
Errors	None.

See also	`getSvLoadLogRecordRawData`
	`SecureSymmetricCryptoTransactionManager.prepareSvGet`
	`SecureSymmetricCryptoTransactionManager.prepareSvDebit`
	`SecureSymmetricCryptoTransactionManager.prepareSvReload`
	`SecureSymmetricCryptoTransactionManager.prepareSvUndebit`

Get SV Load Log Record Raw Data

Signature	`getSvLoadLogRecordRawData() → ByteArray?`
Since	3.0
Description	Returns the raw bytes of the last SV load log record; `null` if not available. The `SvLoadLogRecord` returned by `getSvLoadLogRecord` at the same time is the decoding of this value; both are `null` together.
Returns	The non-empty raw bytes of the record, or `null` if not available.
Pre-conditions	None.
Errors	None.
See also	`getSvLoadLogRecord`

Get Traceability Information

Signature	`getTraceabilityInformation() → ByteArray`
Since	1.1
Description	Traceability information (software issuer ID and discretionary data); empty if not available.
Returns	A `ByteArray`; empty if not available.
Pre-conditions	None.
Errors	None.
See also	`CalypsoCardSelectionExtension.prepareGetData` `TransactionManager.prepareGetData`

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Get Transaction Counter

Signature	`getTransactionCounter() → Int`
Since	1.2
Description	Transaction counter from the output of the last successful **Open Secure Session**. **Note:** commands such as **Change Key**, **Change/Verify PIN**, **SV Debit/Undebit/Reload** decrement the card counter but do not update this returned value.
Returns	An `Int` value.
Pre-conditions	**State**—a secure session is open.
Errors	None.
See also	`CalypsoCardSelectionExtension.preparePreOpenSecureSession` `SecureSymmetricCryptoTransactionManager.prepareOpenSecureSession`

Is DF Invalidated

Signature	`isDfInvalidated() → Boolean`
Since	1.0
Description	Tells whether the current DF is invalidated. The invalidation status is determined either from the response to the **Select Application** command or from the response to a **Select File** (DF) command. For a `PRIME_REVISION_3` card, a `6283h` status word is returned in response to the **Select Application** command when the corresponding DF is invalidated. For older Calypso cards, it may be necessary to execute a **Select File** command in order to determine the invalidation status.
Returns	`true` if the current DF has been invalidated, `false` otherwise.
Pre-conditions	None.
Errors	None.

Is DF Ratified

Signature	`isDfRatified() → Boolean`
Since	1.0
Description	Tells whether the last session has been ratified.
Returns	A `Boolean` value.
Pre-conditions	**State**—a secure session is open.
Errors	None.
See also	`CalypsoCardSelectionExtension.preparePreOpenSecureSession` `SecureSymmetricCryptoTransactionManager.prepareOpenSecureSession`

Is Extended Mode Supported

Signature	`isExtendedModeSupported() → Boolean`
Since	1.0
Description	Indicates whether the Extended Mode is supported. This indication is initially interpreted from the **Application Type** byte, but MAY be updated once the secure session is opened: depending on the type of key used, the extended mode functionalities may not be available (for example with a non-AES key).
Returns	A `Boolean` value.
Pre-conditions	None.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Errors	None.

Is HCE

Signature	`isHce() → Boolean`
Since	1.0
Description	Indicates whether the card is a Calypso HCE.
Returns	A `Boolean` value.
Pre-conditions	None.
Errors	None.

Is PIN Blocked

Signature	`isPinBlocked() → Boolean`
Since	1.0
Description	Indicates whether the PIN is blocked.
Returns	A `Boolean` value.
Pre-conditions	**State**—the PIN has been checked.
Errors	None.
See also	`TransactionManager.prepareCheckPinStatus` `TransactionManager.prepareVerifyPin`

Is PIN Feature Available

Signature	`isPinFeatureAvailable() → Boolean`
Since	1.0
Description	Indicates whether the Calypso PIN feature is available. This indication is interpreted from the **Application Type** byte.
Returns	`true` if the card has the PIN feature, `false` otherwise.
Pre-conditions	None.
Errors	None.

Is PKI Mode Supported

Signature	`isPkiModeSupported() → Boolean`
Since	1.0
Description	Indicates whether Public Key Authentication is supported. This indication is interpreted from the **Application Type** byte.
Returns	A `Boolean` value.
Pre-conditions	None.
Errors	None.

Is Ratification On Deselect Supported

Signature	`isRatificationOnDeselectSupported() → Boolean`
Since	1.0

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Description	Indicates whether ratification happens on deselect (ratification command not required). This indication is interpreted from the **Application Type** byte.
Returns	A `Boolean` value.
Pre-conditions	None.
Errors	None.

Is SV Feature Available

Signature	`isSvFeatureAvailable() → Boolean`
Since	1.0
Description	Indicates whether the SV feature is available. This indication is interpreted from the **Application Type** byte.
Returns	`true` if the card has the Stored Value feature, `false` otherwise.
Pre-conditions	None.
Errors	None.

5.3.3. Calypso Card API Factory

Name	`CalypsoCardApiFactory`
Kind	Interface
Namespace	`calypso.card`
Since	2.0
Purpose	Factory used to obtain instances of every public type provided by the API.

Create Asymmetric Crypto Security Settings

Signature	`createAsymmetricCryptoSecuritySettings(`
	`  cryptoCardTransactionManagerFactory:`
	`AsymmetricCryptoCardTransactionManagerFactory`
	`) → AsymmetricCryptoSecuritySettings`

Since	2.1
Description	Creates the `AsymmetricCryptoSecuritySettings` that carries the asymmetric-key (PKI) cryptographic configuration required by `createSecurePkiModeTransactionManager`. The supplied factory provides the cryptographic module that performs those computations.
Parameters	`cryptoCardTransactionManagerFactory` (`AsymmetricCryptoCardTransactionManagerFactory`) — the factory of the crypto card transaction manager to be used.
Returns	An `AsymmetricCryptoSecuritySettings`.
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — the factory is valid.
Errors	None.

Create Calypso Card Selection Extension

Signature	`createCalypsoCardSelectionExtension() → CalypsoCardSelectionExtension`
Since	2.0
Description	Creates a `CalypsoCardSelectionExtension` that enriches a CNA-TR-API selection scenario with Calypso-specific commands and the optional pre-opening of a secure session.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Returns	A `CalypsoCardSelectionExtension`.
Pre-conditions	None.
Errors	None.

Create Free Transaction Manager

Signature	`createFreeTransactionManager(`
	`  cardReader: CardReader,`
	`  card: CalypsoCard`
	`) → FreeTransactionManager`

Since	2.0
Description	Creates a `FreeTransactionManager` for exchanges with `card` (reached through `cardReader`) that require **no** cryptographic computation—that is, no secure session and no security settings. For secured transactions, use one of the `createSecure*ModeTransactionManager` factories instead.
Parameters	`cardReader` (`CardReader` (CNA-TR-API))—the reader through which the card is reached. `card` (`CalypsoCard`)—the selected card on which to operate the transaction.
Returns	A `FreeTransactionManager`.
Pre-conditions	**Argument**—every parameter is non-`null`.
Errors	None.

Create Secure Extended Mode Transaction Manager

Signature	`createSecureExtendedModeTransactionManager(`
	`  cardReader: CardReader,`
	`  card: CalypsoCard,`
	`  securitySettings: SymmetricCryptoSecuritySettings`
	`) → SecureExtendedModeTransactionManager`

Since	2.0
Description	Creates a `SecureExtendedModeTransactionManager` that runs a symmetric-key secure session in **extended mode**, adding the operations available only on **Calypso Prime Extended** products (e.g. session encryption and early mutual authentication). For other products, use `createSecureRegularModeTransactionManager`. Cryptographic operations rely on the supplied `securitySettings`.
Parameters	`cardReader` (`CardReader` (CNA-TR-API))—the reader through which the card is reached. `card` (`CalypsoCard`)—the selected card on which to operate the transaction. `securitySettings` (`SymmetricCryptoSecuritySettings`)—the security settings to be used.
Returns	A `SecureExtendedModeTransactionManager`.
Pre-conditions	**Argument**—every parameter is non-`null`.
Errors	None.

Create Secure PKI Mode Transaction Manager

Signature	`createSecurePkiModeTransactionManager(`
	`  cardReader: CardReader,`
	`  card: CalypsoCard,`
	`  securitySettings: AsymmetricCryptoSecuritySettings`
	`) → SecurePkiModeTransactionManager`

Since	2.1

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Description	Creates a `SecurePkiModeTransactionManager` that runs a secure session in **PKI mode**, using asymmetric-key (PKI) cryptography. PKI mode is available only if the card declares support for it (`CalypsoCard.isPkiModeSupported() == true`). For symmetric-key sessions, use `createSecureRegularModeTransactionManager` or `createSecureExtendedModeTransactionManager` instead. Cryptographic operations rely on the supplied `securitySettings`.
Parameters	`cardReader` (`CardReader` (CNA-TR-API)) — the reader through which the card is reached. `card` (`CalypsoCard`) — the selected card on which to operate the transaction. `securitySettings` (`AsymmetricCryptoSecuritySettings`) — the security settings to be used.
Returns	A `SecurePkiModeTransactionManager`.
Pre-conditions	**Argument** — every parameter is non-`null`. **State** — the card supports PKI mode (`isPkiModeSupported() == true`).
Errors	None.

Create Secure Regular Mode Transaction Manager

Signature	`createSecureRegularModeTransactionManager(`
	`  cardReader: CardReader,`
	`  card: CalypsoCard,`
	`  securitySettings: SymmetricCryptoSecuritySettings`
	`) → SecureRegularModeTransactionManager`

Since	2.0
Description	Creates a `SecureRegularModeTransactionManager` that runs a symmetric-key secure session in **regular mode**, the baseline mode compatible with every Calypso product. To access **Calypso Prime Extended** features use `createSecureExtendedModeTransactionManager`, or `createSecurePkiModeTransactionManager` for PKI cards. Cryptographic operations rely on the supplied `securitySettings`.
Parameters	`cardReader` (`CardReader` (CNA-TR-API)) — the reader through which the card is reached. `card` (`CalypsoCard`) — the selected card on which to operate the transaction. `securitySettings` (`SymmetricCryptoSecuritySettings`) — the security settings to be used.
Returns	A `SecureRegularModeTransactionManager`.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Create Symmetric Crypto Security Settings

Signature	`createSymmetricCryptoSecuritySettings(`
	`  cryptoCardTransactionManagerFactory: SymmetricCryptoCardTransactionManagerFactory`
	`) → SymmetricCryptoSecuritySettings`

Since	2.0
Description	Creates the `SymmetricCryptoSecuritySettings` that carries the symmetric-key (e.g. SAM-backed) cryptographic configuration required by the regular- and extended-mode secure transaction managers. The supplied factory provides the cryptographic module that performs those computations.
Parameters	`cryptoCardTransactionManagerFactory` (`SymmetricCryptoCardTransactionManagerFactory`) — the factory of the crypto card transaction manager to be used.
Returns	A `SymmetricCryptoSecuritySettings`.
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — the factory is valid.
Errors	None.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5.3.4. Calypso Card Selection Extension

Name	`CalypsoCardSelectionExtension`
Kind	Interface
Namespace	`calypso.card.card`
Extends	`CardSelectionExtension` (CNA-TR-API)
Since	2.0
Purpose	Enriches the CNA-TR-API selection scenario with Calypso-specific commands and pre-opening of a secure session. An instance is obtained via `CalypsoCardApiFactory.createCalypsoCardSelectionExtension`.

By default the selection rejects PRIME revision 3 cards that have been **invalidated**. The application MUST call `acceptInvalidatedCard` to stop ignoring them. For earlier revisions, the application MAY have to handle invalidation itself (typically via a **Select File** on the DF).

For all `prepare*` operations, unless otherwise specified, the following parameter ranges apply:

Parameter	Range
SFI	`[0..30]` (`0` indicates the current EF)
Record number	`[1..250]`
Counter number	`[1..83]`
Counter value	`[0..16777215]`
Offset	`[0..249]`, or `[0..32767]` for binary files
Input data length	`[1..250]`, or `[1..32767]` for binary files

Accept Invalidated Card

Signature	`acceptInvalidatedCard() → Self`
Since	1.0
Description	Requests to accept invalidated cards during the selection stage. **Works only with cards that indicate their invalidation status at the time of selection** (e.g., all PRIME Revision 3+ cards and certain PRIME Revision 2 cards).
Returns	The current instance (`Self`).
Pre-conditions	None.
Errors	None.

Prepare Get Data

Signature	`prepareGetData(tag: GetDataTag) → Self`
Since	1.0
Description	Adds a **Get Data** command for the supplied tag. This is the way to obtain FCI information when it is not supplied directly by **Select Application** (e.g. the OMAPI case). **Caution:** the resulting APDU command is compliant with PRIME revision 3 cards. It may therefore be rejected by some earlier revision cards.
Parameters	`tag` (`GetDataTag`) — the data object tag to read.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Errors	None.

Prepare Pre Open Secure Session

Signature	`preparePreOpenSecureSession(`
	`  writeAccessLevel: WriteAccessLevel`
	`) → Self`

Since	1.7
Description	Adds a specific **Open Secure Session** command attempting a **pre-opening**. Enables a future single-exchange session execution by anticipating the APDU responses. The objective of the pre-opening is to allow the grouping of all the commands of a secure session. It is only relevant for a distributed system where the ticketing processing is done remotely, so that a complete secure session can be carried out in a single exchange between the server and the terminal. To achieve that single exchange, all the data that will have to be read in session MUST first be read locally, outside the session—otherwise additional exchanges take place. The remote ticketing processing then prepares all the commands of the session, from opening to closing, before executing it: `prepareOpenSecureSession(…)`, then the other `prepare*` calls, then `prepareCloseSecureSession()`, and finally a single `processCommands()`. The mechanism is **ineffective** if: • the card or crypto module does not support extended mode; • the session needs intermediate exchanges (PIN, SV, encryption, early mutual authentication, unread session data); • the session opens with a different access level; • an intermediate `processCommands()` is called; • asymmetric cryptography is used.
Parameters	`writeAccessLevel` (`WriteAccessLevel`)—the write access level of the session to pre-open.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`. **State**—no pre-open is already prepared.
Errors	None.

Prepare Read Binary

Signature	`prepareReadBinary(`
	`  sfi: Byte,`
	`  offset: Int,`
	`  nbBytesToRead: Int`
	`) → Self`

Since	1.7
Description	Adds one or more **Read Binary** commands to read all or part of the supplied binary EF. Once processed, the result is available in the `CalypsoCard` if the requested file exists in the file structure of the card and if the offset and the number of bytes to read are valid (**best-effort** mode). **Caution:** the resulting APDU command is compliant with PRIME revision 3 cards. It may therefore be rejected by some earlier revision cards.
Parameters	`sfi`—the SFI of the EF. `offset`—the offset (0 indicates the first byte). `nbBytesToRead`—the number of bytes to read.
Returns	The current instance (`Self`).

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Pre-conditions	**Argument**—every parameter is non-`null`. **Argument**—every supplied argument is within range.
Errors	None.

Prepare Read Counter

Signature	`prepareReadCounter(`
	`  sfi: Byte,`
	`  nbCountersToRead: Int`
	`) → Self`

Since	1.7
Description	Adds a **Read Records** command to read a part of a record of the supplied EF, which SHOULD be a counter file. The record is read up to the supplied counter number, so all preceding counters are also read. Once processed, the result is available in the `CalypsoCard` if the requested file and counter number exist in the file structure of the card (**best-effort** mode). **Caution:** the resulting APDU command is compliant with PRIME revision 3 cards. It may therefore be rejected by some earlier revision cards.
Parameters	`sfi`—the SFI of the EF. `nbCountersToRead`—the number of counters to read.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`. **Argument**—every supplied argument is within range.
Errors	None.

Prepare Read Record

Signature	`prepareReadRecord(`
	`  sfi: Byte,`
	`  recordNumber: Int`
	`) → Self`

Since	1.1
Description	Adds a **Read Records** command for a single record of a linear or cyclic EF. Once processed, the result is available in the `CalypsoCard` if the requested file and record exist in the file structure of the card (**best-effort** mode: the command does not fail if the file or the record is absent). **Caution:** the resulting APDU command is compliant with PRIME revision 3 cards. It may therefore be rejected by some earlier revision cards.
Parameters	`sfi`—the SFI of the EF to read. `recordNumber`—the record number to read.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`. **Argument**—every supplied argument is within range.
Errors	None.

Prepare Select File By Control

Signature	`prepareSelectFileByControl(`
	`  selectFileControl: SelectFileControl`
	`) → Self`

Since	1.0

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Description	Adds a **Select File** command using a navigation control (`FIRST`, `NEXT` or `CURRENT`). **Caution:** the resulting APDU command is compliant with PRIME revision 3 cards. It may therefore be rejected by some earlier revision cards.
Parameters	`selectFileControl` (`SelectFileControl`) — the navigation control to apply.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Prepare Select File By LID

Signature	`prepareSelectFileByLid(lid: Short) → Self`
Since	1.0
Description	Adds a **Select File** command targeting an EF by its LID. Caution: PRIME 3 compatible only. The command fails if the selected file is not an EF.
Parameters	`lid` — the LID of the EF to select.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

5.3.5. Free Transaction Manager

Name	`FreeTransactionManager`
Kind	Interface
Namespace	`calypso.card.transaction`
Extends	`TransactionManager`
Since	2.0
Purpose	Transaction manager that does **not** require any cryptographic computation. An instance is obtained via `CalypsoCardApiFactory.createFreeTransactionManager`.

`FreeTransactionManager` does not add any operation.

5.3.6. Secure Extended Mode Transaction Manager

Name	`SecureExtendedModeTransactionManager`
Kind	Interface
Namespace	`calypso.card.transaction`
Extends	`SecureSymmetricCryptoTransactionManager`
Since	2.0
Purpose	Symmetric-key transaction manager exposing additional operations only available on **Calypso Prime Extended** products. An instance is obtained via `CalypsoCardApiFactory.createSecureExtendedModeTransactionManager`.

Prepare Activate Encryption

Signature	`prepareActivateEncryption() → Self`

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Since	2.0
Description	Requests encryption for all following session commands. This ensures data confidentiality and prevents man-in-the-middle attacks. This command only makes sense in the context of a secure session. Encryption is resource intensive and increases transaction times: it is therefore RECOMMENDED to limit it to the commands that require it. Furthermore, when early mutual authentication is also required, it is RECOMMENDED for performance reasons to place the `prepareEarlyMutualAuthentication` and `prepareActivateEncryption` calls consecutively (in any order).
Returns	The current instance (`Self`).
Pre-conditions	**State**—a secure session is open and the encryption is not already active. **Capability**—the **Manage Secure Session** command is available for this context, that is, both the card and the cryptographic module support the extended mode.
Errors	None.
See also	`SecureExtendedModeTransactionManager.prepareDeactivateEncryption` `SecureExtendedModeTransactionManager.prepareEarlyMutualAuthentication`

Prepare Deactivate Encryption

Signature	`prepareDeactivateEncryption() → Self`
Since	2.0
Description	Requests to stop the encryption, which restores the exchanges with the card to their normal mode. This command only makes sense in the context of a secure session in which encryption of the commands has previously been requested. NOTE: `prepareCloseSecureSession()` automatically stops the encryption.
Returns	The current instance (`Self`).
Pre-conditions	**State**—a secure session is open and the encryption is active. **Capability**—the **Manage Secure Session** command is available for this context, that is, both the card and the cryptographic module support the extended mode.
Errors	None.

See also	`SecureExtendedModeTransactionManager.prepareActivateEncryption`
	`SecureExtendedModeTransactionManager.prepareEarlyMutualAuthentication`
	`SecureTransactionManager.prepareCloseSecureSession`

Prepare Early Mutual Authentication

Signature	`prepareEarlyMutualAuthentication() → Self`
Since	2.0
Description	Requests to mutually authenticate the card and the terminal **before** the secure session is closed. The use of this feature penalises the execution time of the secure session and SHOULD be used only when needed (e.g. before sending sensitive commands).
Returns	The current instance (`Self`).
Pre-conditions	**State**—a secure session is open. **Capability**—the **Manage Secure Session** command is available for this context, that is, both the card and the cryptographic module support the extended mode.
Errors	None.
See also	`SecureExtendedModeTransactionManager.prepareActivateEncryption` `SecureExtendedModeTransactionManager.prepareDeactivateEncryption`

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

5.3.7. Secure PKI Mode Transaction Manager

Name	`SecurePkiModeTransactionManager`
Kind	Interface
Namespace	`calypso.card.transaction`
Extends	`SecureTransactionManager`
Since	2.1
Purpose	Asymmetric-key transaction manager, compatible with Calypso cards in **PKI mode**. An instance is obtained via `CalypsoCardApiFactory.createSecurePkiModeTransactionManager`.

Prepare Open Secure Session

Signature	`prepareOpenSecureSession() → Self`
Since	2.1
Description	Schedules an **Open Secure Session** command in PKI mode. If the next prepared command is a **Read One Record** or **Read One Or More Counters**, it will be merged with the session opening for optimisation.
Returns	The current instance (`Self`).
Pre-conditions	**State** — no secure session is already open.
Errors	None.

5.3.8. Secure Regular Mode Transaction Manager

Name	`SecureRegularModeTransactionManager`
Kind	Interface
Namespace	`calypso.card.transaction`
Extends	`SecureSymmetricCryptoTransactionManager`
Since	2.0
Purpose	Symmetric-key transaction manager compatible with every Calypso product. An instance is obtained via `CalypsoCardApiFactory.createSecureRegularModeTransactionManager`.

No additional operation declared.

5.3.9. Secure Symmetric Crypto Transaction Manager

Name	`SecureSymmetricCryptoTransactionManager`
Kind	Interface
Namespace	`calypso.card.transaction`
Extends	`SecureTransactionManager`
Since	2.0
Purpose	Common operations for every transaction manager backed by **symmetric** cryptography.

Prepare Change Key

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Signature	`prepareChangeKey(`
	`  keyIndex: Int,`
	`  newKif: Byte,`
	`  newKvc: Byte,`
	`  issuerKif: Byte,`
	`  issuerKvc: Byte`
	`) → Self`

Since	2.0
Description	Schedules a **Change Key** command. **MUST be performed outside a secure session.** All KIFs/KVCs MUST be available in the crypto module.
Parameters	`keyIndex` — the index of the key to be replaced (`1` for the issuer key, `2` for the load key, `3` for the debit key). `newKif` — the KIF of the new key. `newKvc` — the KVC of the new key. `issuerKif` — the KIF of the current card's issuer key. `issuerKvc` — the KVC of the current card's issuer key.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Capability** — the **Change Key** command is available for this card. **Argument** — the provided key index is within range. **State** — no secure session is open.
Errors	None.

Prepare Invalidate

Signature	`prepareInvalidate() → Self`
Since	2.0
Description	Schedules an **Invalidate** command, which marks the card application as invalidated, blocking further use until it is rehabilitated. This command is usually executed within a secure session with the `DEBIT` key, depending on the access rights granted to it in the file structure of the card. The resulting DF status is available in the `CalypsoCard` via `isDfInvalidated`.
Returns	The current instance (`Self`).
Pre-conditions	**State** — the card is not already invalidated.
Errors	`SessionBufferOverflow` — if the command would overflow the modifications buffer and the multiple-session mode is not allowed.

Prepare Open Secure Session

Signature	`prepareOpenSecureSession(writeAccessLevel: WriteAccessLevel) → Self`
Since	2.0
Description	Schedules an **Open Secure Session** command. The secure session is opened with the supplied `WriteAccessLevel`, depending on whether the transaction profile is personalisation, reload or debit. If the next prepared command is a **Read One Record** or **Read One Or More Counters**, it is merged with the session opening for optimisation, unless the pre-open mode is active or the read-merge optimisation has been disabled. This mechanism may in some cases be incompatible with the security constraints; it can then be disabled via `SymmetricCryptoSecuritySettings.disableReadOnSessionOpening`.
Parameters	`writeAccessLevel` (`WriteAccessLevel`) — the write access level to be used.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **State** — no secure session opening is already prepared, and no secure session is already open.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Errors	None.
See also	`CalypsoCardSelectionExtension.preparePreOpenSecureSession`

Prepare Rehabilitate

Signature	`prepareRehabilitate() → Self`
Since	2.0
Description	Schedules a **Rehabilitate** command, which clears the card application's invalidation status, making it usable again. This command is usually executed within a secure session with the `PERSONALIZATION` key, depending on the access rights granted to it in the file structure of the card. The resulting DF status is available in the `CalypsoCard` via `isDfInvalidated`.
Returns	The current instance (`Self`).
Pre-conditions	**State**—the card is invalidated.
Errors	`SessionBufferOverflow`—if the command would overflow the modifications buffer and the multiple-session mode is not allowed.

Prepare SV Debit

Signature	`prepareSvDebit(`
	`  amount: Int,`
	`  date: ByteArray,`
	`  time: ByteArray`
	`) → Self`

Since	2.0
Description	Schedules an **SV Debit** command, which decreases the current SV balance by the supplied amount. Amount: `0..32767`. The `date` and `time` data fields are recorded in the SV debit log. The key used is the **debit** key. Once processed, the data is available in the `CalypsoCard` through its dedicated SV data management operations.
Parameters	`amount`—the amount to be subtracted (`0..32767`). `date`—a 2-byte free value. `time`—a 2-byte free value.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`. **Argument**—`amount` is in `0..32767`, and `date` and `time` are 2 bytes long. **State**—the new value is non-negative or negative balances are allowed, no other SV command has been prepared inside the same secure session, and the command immediately follows an **SV Get** command prepared with `SvOperation.DEBIT_UNDEBIT`: no other command, prepared or already processed, comes between them, and the channel has remained open since the **SV Get**.
Errors	`SessionBufferOverflow`—if the command would overflow the modifications buffer and the multiple-session mode is not allowed.

Prepare SV Get

Signature	`prepareSvGet(svOperation: SvOperation) → Self`
Since	2.0
Description	Schedules an **SV Get** command, which reads the Stored Value data (balance and logs) that MUST **immediately** precede any SV debit, undebit or reload: no other command may be prepared between them. Once processed, the data is available in the `CalypsoCard` through its dedicated SV data management operations.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Parameters	`svOperation` (`SvOperation`) — the nature of the intended operation: `DEBIT_UNDEBIT` for a debit or an undebit, `RELOAD` for a reload.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Capability** — the SV feature is available for this card.
Errors	None.

Prepare SV Reload

Signature	`prepareSvReload(`
	`  amount: Int,`
	`  date: ByteArray,`
	`  time: ByteArray,`
	`  free: ByteArray`
	`) → Self`

Since	2.0
Description	Schedules an **SV Reload** command. Amount range: `-8388608..8388607`. The `date`, `time` and `free` data fields are recorded in the SV load log. The key used is the **reload** key. Once processed, the data is available in the `CalypsoCard` through its dedicated SV data management operations.
Parameters	`amount` — the value to be reloaded, positive or negative integer in the range `-8388608..8388607`. `date` — a 2-byte free value. `time` — a 2-byte free value. `free` — a 2-byte free value.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — `amount` is in `-8388608..8388607`, and `date`, `time` and `free` are 2 bytes long. **State** — no other SV command has been prepared inside the same secure session, and the command immediately follows an **SV Get** command prepared with `SvOperation.RELOAD`: no other command, prepared or already processed, comes between them, and the channel has remained open since the **SV Get**.
Errors	`SessionBufferOverflow` — if the command would overflow the modifications buffer and the multiple-session mode is not allowed.

Prepare SV Undebit

Signature	`prepareSvUndebit(`
	`  amount: Int,`
	`  date: ByteArray,`
	`  time: ByteArray`
	`) → Self`

Since	3.0
Description	Schedules an **SV Undebit** command, which cancels, totally or partially, a previous debit by increasing the current SV balance by the supplied amount. Amount: `0..32768`. The `date` and `time` data fields are recorded in the SV debit log. The key used is the **debit** key. Once processed, the data is available in the `CalypsoCard` through its dedicated SV data management operations.
Parameters	`amount` — the amount to be added back (`0..32768`). `date` — a 2-byte free value. `time` — a 2-byte free value.
Returns	The current instance (`Self`).

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — `amount` is in `0..32768`, and `date` and `time` are 2 bytes long. **State** — no other SV command has been prepared inside the same secure session, and the command immediately follows an **SV Get** command prepared with `SvOperation.DEBIT_UNDEBIT`: no other command, prepared or already processed, comes between them, and the channel has remained open since the **SV Get**.
Errors	`SessionBufferOverflow` — if the command would overflow the modifications buffer and the multiple-session mode is not allowed.

5.3.10. Secure Transaction Manager

Name	`SecureTransactionManager`
Kind	Interface
Namespace	`calypso.card.transaction`
Extends	`TransactionManager`
Since	2.0
Purpose	Common operations for every secure (cryptographically-backed) transaction manager.

Get Crypto Extension

Signature	`getCryptoExtension() → CardTransactionCryptoExtension`
Since	2.0
Description	Returns the `CardTransactionCryptoExtension` instance created for this card transaction by the crypto module factory attached to the security settings. Successive calls return the same instance. The caller converts the returned instance to the concrete extension type defined by the crypto module in use. Commands prepared through the crypto extension are queued together with the card commands, in call order, and are processed by the same `processCommands()`. This allows card and crypto commands to be interleaved within a single card transaction.
Returns	The crypto extension of this card transaction.
Pre-conditions	None.
Errors	None.

Prepare Cancel Secure Session

Signature	`prepareCancelSecureSession() → Self`
Since	2.0
Description	Schedules a special **Close Secure Session** command aborting the current secure session. Executed in safe mode — never raises errors.
Returns	The current instance (`Self`).
Pre-conditions	None.
Errors	None.

Prepare Close Secure Session

Signature	`prepareCloseSecureSession() → Self`
Since	2.0

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Schedules a **Close Secure Session** command. The ratification mechanism is disabled by default but MAY be enabled via `SymmetricCryptoSecuritySettings.enableRatificationMechanism`. In this case, a ratification command MUST be added after the **Close Secure Session** command when in contactless mode.
Returns	The current instance (`Self`).
Pre-conditions	**State**—a secure session is open or its opening is prepared, and neither a closing nor a cancelling is already prepared.
Errors	None.

5.3.11. Security Settings

Name	`SecuritySettings`
Kind	Interface
Namespace	`calypso.card.transaction`
Since	3.0
Purpose	Settings shared by every Calypso card secure transaction, whatever the cryptographic nature of the secure session: duration bounds of the bounded command exchanges (see Command exchange duration bounds) and tolerance of a missing file or record inside a secure session. Implemented by `SymmetricCryptoSecuritySettings` and `AsymmetricCryptoSecuritySettings`.

Assign Close Secure Session Max Duration By CSN

Signature	`assignCloseSecureSessionMaxDurationByCsn(`
	`  maxDuration: Long,`
	`  csnMin: Long`
	`) → Self`

Since	3.0
Description	Sets the maximum duration (in microseconds) of the **Close Secure Session** command exchange for the cards whose CSN falls within the range starting at `csnMin`. See Command exchange duration bounds.
Parameters	`maxDuration`—the maximum duration of the **Close Secure Session** command exchange, in microseconds, or `Long.MAX_VALUE` to defer to the FCI-based settings for this range. `csnMin`—the lowest card serial number (CSN) of the range, compared as an unsigned 64-bit value.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`. **Argument**—`maxDuration` is strictly positive.
Errors	None.

Assign Close Secure Session Max Duration By FCI

Signature	`assignCloseSecureSessionMaxDurationByFci(`
	`  maxDuration: Long,`
	`  fciRegex: String`
	`) → Self`

Since	3.0
Description	Sets the maximum duration (in microseconds) of the **Close Secure Session** command exchange for the cards whose FCI matches `fciRegex`. The FCI-based settings are evaluated in declaration order and only when no CSN-based setting applies. When `fciRegex` filters on the startup information data, a `prepareGetData(FCI_FOR_CURRENT_DF)` MUST be scheduled inside the secure session to guarantee the integrity of the FCI obtained during the selection. See Command exchange duration bounds.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Parameters	`maxDuration` — the maximum duration of the **Close Secure Session** command exchange, in microseconds, or `Long.MAX_VALUE` for no bound. `fciRegex` — the regular expression matched against the whole FCI, represented as an uppercase hexadecimal string.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — `maxDuration` is strictly positive. **Argument** — `fciRegex` is a valid regular expression within the portable subset defined by Command exchange duration bounds.
Errors	None.

Assign Open Secure Session Max Duration By CSN

Signature	`assignOpenSecureSessionMaxDurationByCsn(`
	`  maxDuration: Long,`
	`  csnMin: Long`
	`) → Self`

Since	3.0
Description	Sets the maximum duration (in microseconds) of the **Open Secure Session** command exchange for the cards whose CSN falls within the range starting at `csnMin`. See Command exchange duration bounds.
Parameters	`maxDuration` — the maximum duration of the **Open Secure Session** command exchange, in microseconds, or `Long.MAX_VALUE` to defer to the FCI-based settings for this range. `csnMin` — the lowest card serial number (CSN) of the range, compared as an unsigned 64-bit value.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — `maxDuration` is strictly positive.
Errors	None.

Assign Open Secure Session Max Duration By FCI

Signature	`assignOpenSecureSessionMaxDurationByFci(`
	`  maxDuration: Long,`
	`  fciRegex: String`
	`) → Self`

Since	3.0
Description	Sets the maximum duration (in microseconds) of the **Open Secure Session** command exchange for the cards whose FCI matches `fciRegex`. The FCI-based settings are evaluated in declaration order and only when no CSN-based setting applies. When `fciRegex` filters on the startup information data, a `prepareGetData(FCI_FOR_CURRENT_DF)` MUST be scheduled inside the secure session to guarantee the integrity of the FCI obtained during the selection. See Command exchange duration bounds.
Parameters	`maxDuration` — the maximum duration of the **Open Secure Session** command exchange, in microseconds, or `Long.MAX_VALUE` for no bound. `fciRegex` — the regular expression matched against the whole FCI, represented as an uppercase hexadecimal string.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — `maxDuration` is strictly positive. **Argument** — `fciRegex` is a valid regular expression within the portable subset defined by Command exchange duration bounds.
Errors	None.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Authorize File Not Found Error

Signature	`authorizeFileNotFoundError() → Self`
Since	3.0
Description	Allows the card to answer that a file does not exist (status word `6A82h`) to a read or select command executed **inside** a secure session without failing the transaction: the affected command is simply not applied to the `CalypsoCard`, and the session continues. Default: disabled, the session then failing in that case. Outside a secure session, this status word never fails the transaction, whatever this setting. This setting is useful for heterogeneous card fleets, where the presence of a file or of a record cannot be known in advance. See `authorizeRecordNotFoundError()`.
Returns	The current instance (`Self`).
Pre-conditions	None.
Errors	None.

Authorize Record Not Found Error

Signature	`authorizeRecordNotFoundError() → Self`
Since	3.0
Description	Allows the card to answer that a record does not exist (status word `6A83h`) to a read or select command executed **inside** a secure session without failing the transaction: the affected command is simply not applied to the `CalypsoCard`, and the session continues. Default: disabled, the session then failing in that case. Outside a secure session, this status word never fails the transaction, whatever this setting. This setting is useful for heterogeneous card fleets, where the presence of a file or of a record cannot be known in advance. See `authorizeFileNotFoundError()`.
Returns	The current instance (`Self`).
Pre-conditions	None.
Errors	None.

5.3.12. Symmetric Crypto Security Settings

Name	`SymmetricCryptoSecuritySettings`
Kind	Interface
Namespace	`calypso.card.transaction`
Extends	`SecuritySettings`
Since	2.0
Purpose	Security settings carrier for symmetric-key Calypso card transactions (e.g. SAM-backed). An instance is obtained via `CalypsoCardApiFactory.createSymmetricCryptoSecuritySettings`.

Add Authorized Session Key

Signature	`addAuthorizedSessionKey(`
	`  kif: Byte,`
	`  kvc: Byte`
	`) → Self`

Since	2.0
Description	Adds an authorised session key. Default: every key accepted; once at least one key is added, only listed keys are accepted.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Parameters	`kif` — the KIF value. `kvc` — the KVC value.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Add Authorized SV Key

Signature	`addAuthorizedSvKey(`
	`  kif: Byte,`
	`  kvc: Byte`
	`) → Self`

Since	2.0
Description	Adds an authorised Stored Value (SV) key, identified by its KIF/KVC. Default: every SV key accepted; once at least one key is added, only listed keys are accepted.
Parameters	`kif` — the KIF value. `kvc` — the KVC value.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Assign Default KIF

Signature	`assignDefaultKif(`
	`  writeAccessLevel: WriteAccessLevel,`
	`  kif: Byte`
	`) → Self`

Since	2.0
Description	Defines the default KIF for a write access level.
Parameters	`writeAccessLevel` (`WriteAccessLevel`) — the write access level concerned. `kif` — the default KIF to use.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Assign Default KVC

Signature	`assignDefaultKvc(`
	`  writeAccessLevel: WriteAccessLevel,`
	`  kvc: Byte`
	`) → Self`

Since	2.0
Description	Defines the default KVC for a write access level.
Parameters	`writeAccessLevel` (`WriteAccessLevel`) — the write access level concerned. `kvc` — the default KVC to use.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Assign KIF

Signature	`assignKif(`
	`  writeAccessLevel: WriteAccessLevel,`
	`  kvc: Byte,`
	`  kif: Byte`
	`) → Self`

Since	2.0
Description	Defines, for a write access level, the KIF to use when the card only provides a KVC.
Parameters	`writeAccessLevel` (`WriteAccessLevel`) — the write access level concerned. `kvc` — the KVC provided by the card. `kif` — the KIF to associate with that KVC.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Assign SV Command Max Duration By CSN

Signature	`assignSvCommandMaxDurationByCsn(`
	`  maxDuration: Long,`
	`  csnMin: Long`
	`) → Self`

Since	3.0
Description	Sets the maximum duration (in microseconds) of an **SV Reload**, **SV Debit** or **SV Undebit** command exchange for the cards whose CSN falls within the range starting at `csnMin`. See Command exchange duration bounds.
Parameters	`maxDuration` — the maximum duration of an **SV Reload**, **SV Debit** or **SV Undebit** command exchange, in microseconds, or `Long.MAX_VALUE` to defer to the FCI-based settings for this range. `csnMin` — the lowest card serial number (CSN) of the range, compared as an unsigned 64-bit value.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — `maxDuration` is strictly positive.
Errors	None.

Assign SV Command Max Duration By FCI

Signature	`assignSvCommandMaxDurationByFci(`
	`  maxDuration: Long,`
	`  fciRegex: String`
	`) → Self`

Since	3.0
Description	Sets the maximum duration (in microseconds) of an **SV Reload**, **SV Debit** or **SV Undebit** command exchange for the cards whose FCI matches `fciRegex`. The FCI-based settings are evaluated in declaration order and only when no CSN-based setting applies. When `fciRegex` filters on the startup information data, a `prepareGetData(FCI_FOR_CURRENT_DF)` MUST be scheduled inside the secure session to guarantee the integrity of the FCI obtained during the selection. See Command exchange duration bounds.
Parameters	`maxDuration` — the maximum duration of an **SV Reload**, **SV Debit** or **SV Undebit** command exchange, in microseconds, or `Long.MAX_VALUE` for no bound. `fciRegex` — the regular expression matched against the whole FCI, represented as an uppercase hexadecimal string.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — `maxDuration` is strictly positive. **Argument** — `fciRegex` is a valid regular expression within the portable subset defined by Command exchange duration bounds.
Errors	None.

Authorize SV Negative Balance

Signature	`authorizeSvNegativeBalance() → Self`
Since	2.0
Description	Allows the SV balance to become negative. Default: disabled.
Returns	The current instance (`Self`).
Pre-conditions	None.
Errors	None.

Disable Read On Session Opening

Signature	`disableReadOnSessionOpening() → Self`
Since	2.0
Description	Disables the automatic merging of **Open Secure Session** with a possible **Read Record** command. By default, the optimisation is performed; it may however be incompatible with the security requirements in some cases.
Returns	The current instance (`Self`).
Pre-conditions	None.
Errors	None.

Enable Multiple Session

Signature	`enableMultipleSession() → Self`
Since	2.0
Description	Enables multiple-session mode to handle more changes than the session buffer allows. Default: disabled. When a command would overflow the modifications buffer, the current secure session is closed and a new one is opened to carry that command. If the split occurs on an SV debit, undebit or reload, the **SV Get** command it follows MUST be re-issued in the new session, immediately before it, so that the SV command still immediately follows its **SV Get** and remains the only SV command of the session it is executed in.
Returns	The current instance (`Self`).
Pre-conditions	None.
Errors	None.

Enable PIN Plain Transmission

Signature	`enablePinPlainTransmission() → Self`
Since	2.0
Description	Enables the PIN transmission in plain text.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Returns	The current instance (`Self`).
Pre-conditions	None.
Errors	None.

Enable Ratification Mechanism

Signature	`enableRatificationMechanism() → Self`
Since	2.0
Description	Enables the ratification mechanism, which handles an early card removal preventing the terminal from receiving the acknowledgement of the **Close Secure Session** command. Default: disabled. When enabled, a ratification command is added after the closing command in contactless mode. The card considers the secure session **ratified** only once it has processed that command: if the card is removed beforehand, the session stays unratified in the card, which thus keeps the proof that the terminal may not have been informed of the outcome. On cards where ratification happens on deselect, no ratification command is required (see `CalypsoCard.isRatificationOnDeselectSupported`). Whether to enable the mechanism depends on which terminal resumes an interrupted transaction: · **Enable it** when the resumption MAY be handled by **another** terminal, since the proof of the interruption is then carried by the card itself. Typical case: several gates on the same access line of a network. · **Leave it disabled** when the resumption is necessarily handled by the **same** terminal, which keeps the context of the interrupted transaction. Typical case: a remote ticket reloading server. The mechanism can only be used within the anti-passback delay defined by the transport operator: beyond this delay, an unratified session can no longer be handled as the resumption of an interrupted transaction.
Returns	The current instance (`Self`).
Pre-conditions	None.
Errors	None.
See also	`SecureTransactionManager.prepareCloseSecureSession`, `CalypsoCard.isRatificationOnDeselectSupported`

Enable SV Load And Debit Log

Signature	`enableSvLoadAndDebitLog() → Self`
Since	2.0
Description	Enables retrieval of both load **and** debit log records. Default: disabled.
Returns	The current instance (`Self`).
Pre-conditions	None.
Errors	None.

Init Crypto Context For Next Transaction

Signature	`initCryptoContextForNextTransaction() → Unit`
Since	2.0

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Prepares the crypto module for the next transaction by anticipating the security context configuration. This feature is only useful when the currently allocated cryptographic module will be used for the next transaction. It is particularly relevant to optimise the transaction time in a ticketing context of user card validation. For the optimisation to be effective, this operation MUST be called at the very end of the current transaction, that is, **after** the user has been notified of the access right (e.g. after opening the gate).
Pre-conditions	None.
Errors	`Crypto` — if an error occurred when computing a crypto operation. `CryptoIO` — if a communication error with the crypto module occurred (e.g. timeout with the reader or the computing unit, network error, etc.).

Set PIN Modification Ciphering Key

Signature	`setPinModificationCipheringKey(`
	`  kif: Byte,`
	`  kvc: Byte`
	`) → Self`

Since	2.0
Description	Sets the KIF/KVC of the PIN modification ciphering key. Default: 0/0.
Parameters	`kif` — the KIF value. `kvc` — the KVC value.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Set PIN Verification Ciphering Key

Signature	`setPinVerificationCipheringKey(`
	`  kif: Byte,`
	`  kvc: Byte`
	`) → Self`

Since	2.0
Description	Sets the KIF/KVC of the PIN verification ciphering key. Default: 0/0.
Parameters	`kif` — the KIF value. `kvc` — the KVC value.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

5.3.13. Transaction Manager

Name	`TransactionManager`
Kind	Interface
Namespace	`calypso.card.transaction`
Extends	`IsoCardTransactionManager` (CNA-TR-API)
Since	2.0

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Purpose	Common operations for every Calypso card transaction. Prepare/process model. Extending `IsoCardTransactionManager` grants access to the multi-channel cast `asMultichannelCardTransactionManager()` defined by CNA-TR-API when the underlying ISO/IEC 7816-4 card supports multi-channel.

`TransactionManager` defines the full **prepare** command surface (file selection, reads, writes, counters, PIN, asymmetric key-pair generation). The preparation step makes it possible to group commands together in order to minimise network data exchanges, which is especially useful in a distributed architecture. The bound `CalypsoCard` is updated after each successful APDU exchange, and parameter ranges are validated as documented in Section 5.3.4.

Processing of prepared commands is inherited from `CardTransactionManager` (CNA-TR-API) via the `IsoCardTransactionManager` supertype. The Calypso `TransactionManager` does **not** redefine `processCommands()`; it relies on the inherited contract. Multi-channel operations (`processCommandsAndCloseChannel`, `closeChannel`) are reached on demand through `asMultichannelCardTransactionManager()` of CNA-TR-API when the underlying card supports multi-channel.



Every write operation MAY raise `SessionBufferOverflow` if the command would overflow the modification buffer and the multiple-session mode is not allowed.

Get Exchanged APDUs

Signature	`getExchangedApdus() → List<ByteArray>`
Since	1.2
Description	Returns the APDU commands and responses exchanged with the card and with the crypto module since the beginning of the transaction, in chronological order. Intended for logging and diagnosis; the content is not interpreted by the API.  The name `getTransactionAuditData`, used up to version 2.3, is reserved for a future operation returning genuine transaction audit data.
Returns	A `List<ByteArray>`; empty if no exchange has taken place.
Pre-conditions	None.
Errors	None.

Get Secure Session State

Signature	`getSecureSessionState() → SecureSessionState`
Since	3.0
Description	Returns the state of the secure session at the moment of the call.
Returns	A `SecureSessionState` value; `NO_SESSION` if no secure session is open.
Pre-conditions	None.
Errors	None.

Prepare Append Record

Signature	`prepareAppendRecord(`
	`  sfi: Byte,`
	`  recordData: ByteArray`
	`) → Self`

Since	1.0

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Adds an **Append Record** command: a new record is added, and the oldest record is deleted in a cyclic file. Once processed, the data is available in the `CalypsoCard` through its dedicated file and data management operations.
Parameters	`sfi` — the SFI of the EF. `recordData` — the new record data to write.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — every supplied argument is within range.
Errors	`SessionBufferOverflow` — if the command would overflow the modifications buffer of a secure session open in symmetric crypto mode and the multiple-session mode is not allowed.

Prepare Change PIN

Signature	`prepareChangePin(newPin: ByteArray) → Self`
Since	1.6
Description	Adds a **Change PIN** command. It MUST be performed outside a secure session. If transmitted plain, the command MUST be preceded by a successful **Verify PIN**. Once processed, the PIN status is available in the `CalypsoCard` through `getPinAttemptRemaining` and `isPinBlocked`.
Parameters	`newPin` — the new PIN code value (4-byte long `ByteArray`).
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Capability** — the PIN feature is available for this card. **Argument** — the provided argument is within range. **State** — no secure session is open.
Errors	None.

Prepare Check PIN Status

Signature	`prepareCheckPinStatus() → Self`
Since	1.0
Description	Adds a **Verify PIN** command without PIN presentation, in order to read the attempt counter. Once processed, the PIN status is available in the `CalypsoCard` through `getPinAttemptRemaining` and `isPinBlocked`.
Returns	The current instance (`Self`).
Pre-conditions	**Capability** — the PIN feature is available for this card.
Errors	None.

Prepare Decrease Counter

Signature	`prepareDecreaseCounter(`
	`  sfi: Byte,`
	`  counterNumber: Int,`
	`  decValue: Int`
	`) → Self`

Since	1.0

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Adds a **Decrease** command, which subtracts the supplied value from the designated counter of the selected EF. If several counters of the same file have to be decremented at the same time of the transaction, it is RECOMMENDED to use `prepareDecreaseCounters` for optimisation reasons. Once processed, the data is available in the `CalypsoCard` through its dedicated file and data management operations.
Parameters	`sfi` — the SFI of the EF. `counterNumber` — the number of the counter (MUST be zero in case of a simulated counter). `decValue` — the value to subtract from the counter (positive integer ⇐ `16777215`).
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — every supplied argument is within range.
Errors	`SessionBufferOverflow` — if the command would overflow the modifications buffer of a secure session open in symmetric crypto mode and the multiple-session mode is not allowed.

Prepare Decrease Counters

Signature	`prepareDecreaseCounters(`
	`  sfi: Byte,`
	`  decrementValues: Map<Int, Int>`
	`) → Self`

Since	1.1
Description	Adds a **Decrease Multiple** command, or multiple **Decrease** commands. The decision to execute one or the other is made according to the type of card. Once processed, the data is available in the `CalypsoCard` through its dedicated file and data management operations.
Parameters	`sfi` — the SFI of the EF. `decrementValues` — the decrement values to apply, indexed by the numbers of the counters to be decremented.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	`SessionBufferOverflow` — if the command would overflow the modifications buffer of a secure session open in symmetric crypto mode and the multiple-session mode is not allowed.

Prepare Generate Asymmetric Key Pair

Signature	`prepareGenerateAsymmetricKeyPair() → Self`
Since	2.1
Description	Adds a **Generate Asymmetric Key Pair** command. The public part can be retrieved via `prepareGetData(GetDataTag.CARD_PUBLIC_KEY)`.
Returns	The current instance (`Self`).
Pre-conditions	**State** — no secure session is open. **Capability** — the card supports the PKI mode.
Errors	None.

Prepare Get Data

Signature	`prepareGetData(tag: GetDataTag) → Self`
Since	1.0

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Adds one or more **Get Data** commands. **Security warning:** this command MUST NOT be used within a secure session, in both contact and contactless modes, except with the tag `FCI_FOR_CURRENT_DF`. With the tag `FCI_FOR_CURRENT_DF` inside a secure session, the command is used to check the integrity of the FCI obtained in response to the **Select Application** command: the following `processCommands()` raises `InconsistentData` if the returned FCI differs from that response. Once processed, the data is available in the `CalypsoCard` through `ElementaryFile.header` or `getDirectoryHeader`, depending on the supplied tag.
Parameters	`tag` (`GetDataTag`) — the data type to retrieve.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Capability** — the **Get Data** command is supported for the provided tag. **State** — no secure session is open, unless `tag` is `FCI_FOR_CURRENT_DF`.
Errors	None.

Prepare Increase Counter

Signature	`prepareIncreaseCounter(`
	`  sfi: Byte,`
	`  counterNumber: Int,`
	`  incValue: Int`
	`) → Self`

Since	1.0
Description	Adds an **Increase** command. If several counters of the same file have to be incremented at the same time of the transaction, it is RECOMMENDED to use `prepareIncreaseCounters` for optimisation reasons. Once processed, the data is available in the `CalypsoCard` through its dedicated file and data management operations.
Parameters	`sfi` — the SFI of the EF. `counterNumber` — the number of the counter (MUST be zero in case of a simulated counter). `incValue` — the value to add to the counter (positive integer `⇐ 16777215`).
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — every supplied argument is within range.
Errors	`SessionBufferOverflow` — if the command would overflow the modifications buffer of a secure session open in symmetric crypto mode and the multiple-session mode is not allowed.

Prepare Increase Counters

Signature	`prepareIncreaseCounters(`
	`  sfi: Byte,`
	`  incrementValues: Map<Int, Int>`
	`) → Self`

Since	1.1
Description	Adds an **Increase Multiple** command, or multiple **Increase** commands. The decision to execute one or the other is made according to the type of card. Once processed, the data is available in the `CalypsoCard` through its dedicated file and data management operations.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Parameters	`sfi` — the SFI of the EF. `incrementValues` — the increment values to apply, indexed by the numbers of the counters to be incremented.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	`SessionBufferOverflow` — if the command would overflow the modifications buffer of a secure session open in symmetric crypto mode and the multiple-session mode is not allowed.

Prepare Put Data

Signature	`preparePutData(`
	`  tag: PutDataTag,`
	`  data: ByteArray`
	`) → Self`

Since	2.1
Description	Adds one or more **Put Data** commands. The secure session requirement depends on the tag: • `CARD_KEY_PAIR`, `CARD_CERTIFICATE` and `CA_CERTIFICATE`: the command MUST be prepared **outside** a secure session; • `APPLICATION_AID` and `STORED_VALUE_LINK`: the command MUST be prepared **within** a secure session opened in symmetric crypto mode, that is while `getSecureSessionState()` returns one of the `SYMMETRIC_*` values. The card additionally requires the session to be opened with the `PERSONALIZATION` write access level (issuer key); otherwise it rejects the command and the following `processCommands()` fails. The expected data also depends on the tag: the new AID (5 to 16 bytes) for `APPLICATION_AID`, and the AID of the Stored Value application to link (5 to 16 bytes) or an empty `ByteArray` for `STORED_VALUE_LINK`.
Parameters	`tag` (`PutDataTag`) — the data type to inject. `data` — the data to inject.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Capability** — the **Put Data** command is supported for the provided tag. **Argument** — the length of `data` is consistent with the supplied tag. **State** — no secure session is open, except for `APPLICATION_AID` and `STORED_VALUE_LINK`, which require a secure session opened in symmetric crypto mode.
Errors	`SessionBufferOverflow` — with the tags `APPLICATION_AID` and `STORED_VALUE_LINK`, if the command would overflow the modifications buffer of the secure session and the multiple-session mode is not allowed.

Prepare Read Binary

Signature	`prepareReadBinary(`
	`  sfi: Byte,`
	`  offset: Int,`
	`  nbBytesToRead: Int`
	`) → Self`

Since	1.1

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Adds **Read Binary** commands. The following `processCommands()` does not fail if the targeted file is not found (status word `6A82h`) when the command is placed outside a secure session. Inside a secure session, it fails unless `authorizeFileNotFoundError()` has been called on the security settings; the `CalypsoCard` may then simply not be filled. For the validity of the offset and the number of bytes to read, two behaviours apply depending on whether the command is placed inside a secure session. Outside a secure session (**best-effort** mode), it does not fail whatever their validity. Inside a secure session (**strict** mode), it fails if they are invalid. Once processed, the data is available in the `CalypsoCard` through its dedicated file and data management operations.
Parameters	`sfi` — the SFI of the EF. `offset` — the offset (`0` indicates the first byte). `nbBytesToRead` — the number of bytes to read.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Capability** — this command is supported by this card. **Argument** — every supplied argument is within range.
Errors	None.

Prepare Read Counter

Signature	`prepareReadCounter(`
	`  sfi: Byte,`
	`  nbCountersToRead: Int`
	`) → Self`

Since	1.1
Description	Adds a **Read Records** command targeting the supplied counter. The record is read up to the counter location supplied as parameter, so all preceding counters are also read. The following `processCommands()` does not fail if the targeted file is not found (status word `6A82h`) when the command is placed outside a secure session. Inside a secure session, it fails unless `authorizeFileNotFoundError()` has been called on the security settings; the `CalypsoCard` may then simply not be filled. For the existence of the targeted counter, the same distinction applies: outside a secure session (**best-effort** mode), the command does not fail; inside a secure session, it fails unless `authorizeRecordNotFoundError()` has been called on the security settings. Once processed, the data is available in the `CalypsoCard` through its dedicated file and data management operations.
Parameters	`sfi` — the SFI of the EF. `nbCountersToRead` — the number of counters to read.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — every supplied argument is within range.
Errors	None.

Prepare Read Record

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Signature	`prepareReadRecord(`
	`  sfi: Byte,`
	`  recordNumber: Int`
	`) → Self`

Since	1.1
Description	Adds a **Read Records** command for a single record. **Security warning:** this command MUST NOT be used within a secure session, in both contact and contactless modes. For the in-session case, use `prepareReadRecords` instead, with valid parameters. Once processed, the data is available in the `CalypsoCard` through its dedicated file and data management operations. The following `processCommands()` does not fail whatever the existence of the targeted file or record: the `CalypsoCard` may then simply not be filled (**best-effort** mode).
Parameters	`sfi` —the SFI of the EF. `recordNumber` —the record to read.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`. **Argument**—every supplied argument is within range.
Errors	None.

Prepare Read Records

Signature	`prepareReadRecords(`
	`  sfi: Byte,`
	`  fromRecordNumber: Int,`
	`  toRecordNumber: Int,`
	`  recordSize: Int`
	`) → Self`

Since	1.1
Description	Adds a **Read Records** command for one or more records. The following `processCommands()` does not fail if the targeted file is not found (status word `6A82h`) when the command is placed outside a secure session. Inside a secure session, it fails unless `authorizeFileNotFoundError()` has been called on the security settings; the `CalypsoCard` may then simply not be filled. For the existence of the targeted record, the same distinction applies: outside a secure session (**best-effort** mode), the command does not fail; inside a secure session, it fails unless `authorizeRecordNotFoundError()` has been called on the security settings. Once processed, the data is available in the `CalypsoCard` through its dedicated file and data management operations.
Parameters	`sfi` —the SFI of the EF. `fromRecordNumber`—the number of the first record to read. `toRecordNumber` —the number of the last record to read. `recordSize` —the record length.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`. **Argument**—every supplied argument is within range.
Errors	None.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Prepare Read Records Partially

Signature	`prepareReadRecordsPartially(`
	`  sfi: Byte,`
	`  fromRecordNumber: Int,`
	`  toRecordNumber: Int,`
	`  offset: Int,`
	`  nbBytesToRead: Int`
	`) → Self`

Since	1.1
Description	Adds **Read Record Multiple** commands. **Security warning:** this command MUST NOT be used within a secure session, in both contact and contactless modes. The following `processCommands()` does not fail whatever the existence of the targeted file or the validity of the offset and the number of bytes to read: the `CalypsoCard` may then simply not be filled (**best-effort** mode). Once processed, the data is available in the `CalypsoCard` through its dedicated file and data management operations.
Parameters	`sfi` — the SFI of the EF. `fromRecordNumber` — the number of the first record to read. `toRecordNumber` — the number of the last record to read. `offset` — the offset in the records where to start reading (`0` indicates the first byte). `nbBytesToRead` — the number of bytes to read from each record.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Capability** — this command is supported by this card. **Argument** — every provided argument is within range. **State** — no secure session is open.
Errors	None.

Prepare Search Records

Signature	`prepareSearchRecords(`
	`  commandId: Int,`
	`  data: SearchCommandData`
	`) → Self`

Since	1.1
Description	Adds a **Search Record Multiple** command. The command searches whether the supplied data are present in the records of the file; an optional mask MAY be applied to specify precisely the bits to take into account in the comparison. It is only possible with a **linear**, **cyclic**, **Counters** or **Simulated Counter** EF. **Security warning:** this command MUST NOT be used within a secure session, in both contact and contactless modes. Once processed, the numbers of the matching records are available in the `CalypsoCard` through `getMatchingRecordNumbers` called with `commandId`, and the content of the first matching record in the same `CalypsoCard` when requested. The following `processCommands()` does not fail whatever the existence of the targeted file or the validity of the record number and offset: the `CalypsoCard` may then simply not be updated (**best-effort** mode).
Parameters	`commandId` — the application-supplied identifier of this command. `data` (`SearchCommandData`) — the search criteria.
Returns	The current instance (`Self`).

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Pre-conditions	**Argument**—every parameter is non-`null`. **Capability**—the **Search Record Multiple** command is available for this card. **Argument**—the input data is consistent. **State**—no secure session is open.
Errors	None.
See also	`SearchCommandData`, `CalypsoCard.getMatchingRecordNumbers`

Prepare Select File By Control

Signature	`prepareSelectFileByControl(selectFileControl: SelectFileControl) → Self`
Since	1.0
Description	Adds a **Select File** command using a navigation control. The following `processCommands()` does not fail if the targeted file is not found (status word `6A82h`) when the command is placed outside a secure session. Inside a secure session, it fails unless `authorizeFileNotFoundError()` has been called on the security settings; the `CalypsoCard` may then simply not be updated. Once processed, the data is available in the `CalypsoCard` through `ElementaryFile.header`.
Parameters	`selectFileControl` (`SelectFileControl`)—a `SelectFileControl` enum entry.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`.
Errors	None.

Prepare Select File By LID

Signature	`prepareSelectFileByLid(lid: Short) → Self`
Since	1.1
Description	Adds a **Select File** command targeting an EF by LID. **Caution:** the command fails if the selected file is not an EF. The following `processCommands()` does not fail if the targeted file is not found (status word `6A82h`) when the command is placed outside a secure session. Inside a secure session, it fails unless `authorizeFileNotFoundError()` has been called on the security settings; the `CalypsoCard` may then simply not be updated. Once processed, the data is available in the `CalypsoCard` through `getFileBySfi` / `getFileByLid` and `ElementaryFile.header`.
Parameters	`lid`—the LID of the EF to select.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`.
Errors	None.

Prepare Set Counter

Signature	`prepareSetCounter(`
	`  sfi: Byte,`
	`  counterNumber: Int,`
	`  newValue: Int`
	`) → Self`

Since	1.0

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Description	Adds either an **Increase** or a **Decrease** command to set the counter to the supplied value. The operation is selected according to whether the difference between the current value and the desired value is negative (**Increase**) or positive (**Decrease**). Two assumptions are made, and neither is checked by this operation: the counter value has been read beforehand, and the type of session—and its associated access rights—is consistent with the requested operation (a reload session if the counter is to be incremented, a debit session if it is to be decremented). An inconsistency is only detected when the session is closed. Once processed, the data is available in the `CalypsoCard` through its dedicated file and data management operations.
Parameters	`sfi` —the SFI of the EF. `counterNumber`—the counter number (`>= 1` for a counters file, `0` for a simulated "counter" file). `newValue` —the desired value for the counter (positive integer `<= 16777215`).
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`. **State**—the current counter value is known. **Argument**—every supplied argument is within range.
Errors	`SessionBufferOverflow`—if the command would overflow the modifications buffer of a secure session open in symmetric crypto mode and the multiple-session mode is not allowed.

Prepare SV Read All Logs

Signature	`prepareSvReadAllLogs() → Self`
Since	1.0
Description	Schedules the reading of every Stored Value log, that is, both the load and the debit log records. This operation requires the selected application to be of type **Store Value** (file structure `20h`). The SV transaction logs are contained in two files with fixed identifiers: the file whose SFI is `14h` holds one record carrying the unique reload log, and the file whose SFI is `15h` holds three records carrying the last three debit logs. Once processed, the data is available in the `CalypsoCard` in raw format through its dedicated file and data management operations, or as dedicated objects through `getSvLoadLogRecord` and `getSvDebitLogAllRecords`.
Returns	The current instance (`Self`).
Pre-conditions	**Capability**—the SV feature is available for this card.
Errors	None.

Prepare Update Binary

Signature	`prepareUpdateBinary(`
	`  sfi: Byte,`
	`  offset: Int,`
	`  data: ByteArray`
	`) → Self`

Since	1.1
Description	Adds one or more **Update Binary** commands. The data of the file before the offset and after the supplied data are left unchanged. Once processed, the data is available in the `CalypsoCard` through its dedicated file and data management operations.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Parameters	`sfi` — the SFI of the EF. `offset` — the offset (`0` indicates the first byte). `data` — the new data.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Capability** — this command is supported by this card. **Argument** — every supplied argument is within range.
Errors	`SessionBufferOverflow` — if the command would overflow the modifications buffer of a secure session open in symmetric crypto mode and the multiple-session mode is not allowed.

Prepare Update Record

Signature	`prepareUpdateRecord(`
	`  sfi: Byte,`
	`  recordNumber: Int,`
	`  recordData: ByteArray`
	`) → Self`

Since	1.0
Description	Adds an **Update Record** command. Bytes beyond `recordData.length` are left unchanged. Once processed, the data is available in the `CalypsoCard` through its dedicated file and data management operations.
Parameters	`sfi` — the SFI of the EF. `recordNumber` — the record to update. `recordData` — the new record data. If length is less than the record size, bytes beyond length are left unchanged.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — every supplied argument is within range.
Errors	`SessionBufferOverflow` — if the command would overflow the modifications buffer of a secure session open in symmetric crypto mode and the multiple-session mode is not allowed.

Prepare Verify PIN

Signature	`prepareVerifyPin(pin: ByteArray) → Self`
Since	1.6
Description	Adds a **Verify PIN** command. This command MAY be performed both inside and outside a secure session. The PIN code is transmitted in plain text or enciphered, according to the parameter set in the `SymmetricCryptoSecuritySettings`. Once processed, the PIN status is available in the `CalypsoCard` through `getPinAttemptRemaining` and `isPinBlocked`.
Parameters	`pin` — the PIN code value (4-byte long `ByteArray`).
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Capability** — the PIN feature is available for this card. **Argument** — the provided argument is within range.
Errors	None.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Prepare Write Binary

Signature	`prepareWriteBinary(`
	`  sfi: Byte,`
	`  offset: Int,`
	`  data: ByteArray`
	`) → Self`

Since	1.1
Description	Adds one or more **Write Binary** commands, performing a binary OR with the existing data. The data of the file before the offset and after the supplied data are left unchanged. Once processed, the data is available in the `CalypsoCard` through its dedicated file and data management operations.
Parameters	`sfi` —the SFI of the EF. `offset`—the offset (`0` indicates the first byte). `data` —the data to write over the existing data.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`. **Capability**—this command is supported by this card. **Argument**—every supplied argument is within range.
Errors	`SessionBufferOverflow`—if the command would overflow the modifications buffer of a secure session open in symmetric crypto mode and the multiple-session mode is not allowed.

Prepare Write Record

Signature	`prepareWriteRecord(`
	`  sfi: Byte,`
	`  recordNumber: Int,`
	`  recordData: ByteArray`
	`) → Self`

Since	1.0
Description	Adds a **Write Record** command, performing a binary OR with the existing data. Once processed, the data is available in the `CalypsoCard` through its dedicated file and data management operations.
Parameters	`sfi` —the SFI of the EF. `recordNumber`—the record to write. `recordData` —the data to overwrite in the record. If length is less than the record size, bytes beyond length are left unchanged.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`. **Argument**—every supplied argument is within range.
Errors	`SessionBufferOverflow`—if the command would overflow the modifications buffer of a secure session open in symmetric crypto mode and the multiple-session mode is not allowed.

5.4. SPI Interfaces

5.4.1. Asymmetric Crypto Card Transaction Manager Factory

Name	`AsymmetricCryptoCardTransactionManagerFactory`
Kind	Marker interface (SPI)

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Namespace	`calypso.card.transaction.spi`
Since	2.0
Purpose	Marker provided by crypto extensions to secure Calypso card transactions with **asymmetric** keys (PKI). Declares no operation.

5.4.2. CA Certificate

Name	`CaCertificate`
Kind	Marker interface (SPI)
Namespace	`calypso.card.transaction.spi`
Since	2.1
Purpose	Marker for a CA certificate exposed by a CNA-TCCA-API extension. Registered through `AsymmetricCryptoSecuritySettings.addCaCertificate`.

5.4.3. CA Certificate Parser

Name	`CaCertificateParser`
Kind	Marker interface (SPI)
Namespace	`calypso.card.transaction.spi`
Since	2.1
Purpose	Marker for a CA certificate parser exposed by a CNA-TCCA-API extension. Registered through `AsymmetricCryptoSecuritySettings.addCaCertificateParser`.

5.4.4. Card Certificate

Name	`CardCertificate`
Kind	Marker interface (SPI)
Namespace	`calypso.card.transaction.spi`
Since	2.1
Purpose	Marker for a card certificate exposed by a CNA-TCCA-API extension. Consumed by the asymmetric crypto extension when authenticating a card; it is not registered directly through `AsymmetricCryptoSecuritySettings`.

5.4.5. Card Certificate Parser

Name	`CardCertificateParser`
Kind	Marker interface (SPI)
Namespace	`calypso.card.transaction.spi`
Since	2.1
Purpose	Marker for a card certificate parser exposed by a CNA-TCCA-API extension. Registered through `AsymmetricCryptoSecuritySettings.addCardCertificateParser`.

5.4.6. Card Transaction Crypto Extension

Name	`CardTransactionCryptoExtension`
Kind	Marker interface (SPI)

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Namespace	`calypso.card.transaction.spi`
Since	2.0
Purpose	Marker enriching the card transaction command set with specific crypto commands (e.g. signature computation/verification). Declares no operation at the API level.

5.4.7. PCA Certificate

Name	`PcaCertificate`
Kind	Marker interface (SPI)
Namespace	`calypso.card.transaction.spi`
Since	2.1
Purpose	Marker for a self-signed PCA certificate exposed by a CNA-TCCA-API extension. Registered through `AsymmetricCryptoSecuritySettings.addPcaCertificate`.

`PcaCertificate`, `CaCertificate` and `CardCertificate` declare no operation at the API level. Their concrete content is exposed through the CNA-TCCA-API extension that produced them.

5.4.8. Symmetric Crypto Card Transaction Manager Factory

Name	`SymmetricCryptoCardTransactionManagerFactory`
Kind	Marker interface (SPI)
Namespace	`calypso.card.transaction.spi`
Since	2.0
Purpose	Marker provided by crypto extensions to secure Calypso card transactions with **symmetric** keys (e.g. SAM). Declares no operation.

5.5. Enumerations

5.5.1. Calypso Card Product Type

Name	`CalypsoCardProductType`
Kind	Enumeration
Namespace	`calypso.card.card`
Since	1.0
Purpose	Product type of a Calypso card.

Value	Since	Description
`PRIME_REVISION_1`	1.0	Calypso Prime revision 1.x.
`PRIME_REVISION_2`	1.0	Calypso Prime revision 2.x.
`PRIME_REVISION_3`	1.0	Calypso Prime revision 3.x.
`LIGHT`	1.0	Calypso Light.
`BASIC`	1.0	Calypso Basic.
`UNKNOWN`	1.0	Application Type is `0` or `FFh`, or selection data could not be properly parsed.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5.5.2. Elementary File Type

Name	`ElementaryFileType`
Kind	Enumeration
Namespace	`calypso.card.card`
Since	1.0
Purpose	Type of a Calypso Elementary File.

Value	Since	Description
`LINEAR`	1.0	Linear EF (1 to several records).
`BINARY`	1.0	Binary EF (single continuous sequence of data bytes).
`CYCLIC`	1.0	Cyclic EF (records organised in a cycle, from the most recent to the oldest).
`COUNTERS`	1.0	Counters EF (single record containing K counters of three bytes each).
`SIMULATED_COUNTERS`	1.0	Simulated counter EF (linear file with a single record, kept for Calypso Rev 2 compatibility).

5.5.3. Get Data Tag

Name	`GetDataTag`
Kind	Enumeration
Namespace	`calypso.card`
Since	1.0
Purpose	Output data tags retrievable through the **Get Data** command. MAY NOT be applicable to all products.

Value	Since	Description
`FCP_FOR_CURRENT_FILE`	1.0	FCP for the current file, as returned by **Select File**.
`FCI_FOR_CURRENT_DF`	1.0	FCI for the current DF, as returned by **Select Application**.
`EF_LIST`	1.1	List of EFs in the current DF.
`TRACEABILITY_INFORMATION`	1.1	Product traceability information.
`CARD_PUBLIC_KEY`	2.1	Card public key.
`CARD_CERTIFICATE`	2.1	Card certificate.
`CA_CERTIFICATE`	2.1	CA certificate.

5.5.4. Put Data Tag

Name	`PutDataTag`
Kind	Enumeration
Namespace	`calypso.card`
Since	2.1
Purpose	Input data tags injectable through the **Put Data** command. MAY NOT be applicable to all products.

Value	Since	Description
`CARD_KEY_PAIR`	2.1	Card key pair.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Value	Since	Description
`CARD_CERTIFICATE`	2.1	Card certificate.
`CA_CERTIFICATE`	2.1	CA certificate.
`APPLICATION_AID`	2.3	New AID of the current application (tag `4Fh`). The data is the new AID value (5 to 16 bytes). The new AID applies to the next selections of the application; it does not change the DF name already exposed by the `CalypsoCard`.
`STORED_VALUE_LINK`	2.3	Link of the current application to a Stored Value application of the same card (tag `D0h`). The data is the AID of the Stored Value application to link (5 to 16 bytes), or an empty `ByteArray` to cancel the existing link.

5.5.5. Secure Session State

Name	`SecureSessionState`
Kind	Enumeration
Namespace	`calypso.card.transaction`
Since	3.0
Purpose	State of the secure session, returned by `TransactionManager.getSecureSessionState`.

The value reflects the state of the secure session at the moment of the call; a new call to `getSecureSessionState` is required to observe later changes. A secure session is open if and only if the value differs from `NO_SESSION`.



The values reflect the **cryptographic nature** of the session and not the **application mode** (Regular / Extended). Both symmetric modes share the same underlying cryptography and only differ in their configuration at the `TransactionManager` level; the integrator who needs to know the application mode obtains it directly via the sub-type of `TransactionManager` that has been instantiated.

Value	Since	Description
`NO_SESSION`	3.0	No secure session is open: none has been opened yet, or the last one has been closed or cancelled.
`ASYMMETRIC`	3.0	PKI secure session.
`SYMMETRIC_PERSONALIZATION`	3.0	Symmetric secure session opened with the personalisation write access level.
`SYMMETRIC_LOAD`	3.0	Symmetric secure session opened with the load write access level.
`SYMMETRIC_DEBIT`	3.0	Symmetric secure session opened with the debit write access level.

5.5.6. Select File Control

Name	`SelectFileControl`
Kind	Enumeration
Namespace	`calypso.card`
Since	1.0
Purpose	Expected behaviour of the **Select File** command (cf. ISO/IEC 7816-4 and Calypso specifications).

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Value	Since	Description
`FIRST_EF`	1.0	The first EF of the current Calypso DF.
`NEXT_EF`	1.0	The next EF of the current Calypso DF.
`CURRENT_DF`	1.0	The current Calypso DF.

5.5.7. SV Operation

Name	`SvOperation`
Kind	Enumeration
Namespace	`calypso.card.transaction`
Since	1.0
Purpose	SV operation announced by the **SV Get** command: reload, or debit/undebit.

Value	Since	Description
`RELOAD`	1.0	Reload operation: increase the SV balance.
`DEBIT_UNDEBIT`	1.0	Debit or undebit operation: decrease the SV balance, or cancel a previous debit.

5.5.8. Write Access Level

Name	`WriteAccessLevel`
Kind	Enumeration
Namespace	`calypso.card`
Since	1.0
Purpose	Write access level for a Calypso secure session. Each level induces the use of a different session-key role.

Value	Since	Description
`PERSONALIZATION`	1.0	Personalisation, load and debit operations — uses the **issuer** key.
`LOAD`	1.0	Load and debit operations — uses the **load** key.
`DEBIT`	1.0	Debit operations only — uses the **debit** key.

5.6. Errors

Every error defined below carries two properties: `message` (`String`), which describes the condition, and `cause` (`Any?`), the underlying error that triggered it, `null` when there is none. They are not restated for each error.

5.6.1. Card Signature Not Verifiable

Name	`CardSignatureNotVerifiable`
Kind	Error
Namespace	`calypso.card.transaction`
Since	1.2
Purpose	Indicates that the card has correctly closed the secure session but that the authenticity of the session cannot be verified because the crypto module is no longer available (timeout, network problem, etc.).

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5.6.2. Crypto

Name	`Crypto`
Kind	Error
Namespace	`calypso.card.transaction`
Since	2.0
Purpose	Indicates that an error occurred when computing a crypto operation.

5.6.3. Crypto IO

Name	`CryptoIO`
Kind	Error
Namespace	`calypso.card.transaction`
Since	2.0
Purpose	Indicates a communication error with the crypto module.

5.6.4. Inconsistent Data

Name	`InconsistentData`
Kind	Error
Namespace	`calypso.card.transaction`
Since	1.2
Purpose	Indicates the detection of inconsistent data.

The inconsistency falls into one of the following cases:

- a **de-synchronisation of the APDU exchanges**, that is, a number of APDU responses different from the number of APDU requests;
- an **inconsistency in the card data**, which can happen for example when the data read outside the secure session differs from the data read inside it.

5.6.5. Invalid Card Signature

Name	`InvalidCardSignature`
Kind	Error
Namespace	`calypso.card.transaction`
Since	1.2
Purpose	Indicates that the card signature is incorrect. For a transaction secured by **symmetric** cryptography (e.g. SAM), this indicates that the card has correctly closed the secure session, but that the card session is **not authentic** because the MAC it produced is wrong. This typically happens when the **Digest Authenticate** or **SV Check** status word is `6988h`. For a transaction secured by **asymmetric** cryptography (e.g. PKI), this indicates only that the card signature is incorrect.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5.6.6. Invalid Certificate

Name	`InvalidCertificate`
Kind	Error
Namespace	`calypso.card.transaction`
Since	2.1
Purpose	Indicates that a certificate failed validation (signature, validity period, issuer or subject details, constraints, extensions, expiration, revocation).

5.6.7. Invalid PIN

Name	`InvalidPin`
Kind	Error
Namespace	`calypso.card.transaction`
Since	2.0
Purpose	Indicates that the provided PIN is invalid.

5.6.8. Session Buffer Overflow

Name	`SessionBufferOverflow`
Kind	Error
Namespace	`calypso.card.transaction`
Since	1.2
Purpose	Indicates that the secure session cannot be performed atomically because the modification buffer capacity is insufficient.

5.6.9. Unauthorized Key

Name	`UnauthorizedKey`
Kind	Error
Namespace	`calypso.card.transaction`
Since	1.0
Purpose	Indicates that the card requires an unauthorised session key.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement